dash0

DEVELOPER RELATIONS · AUGUST 2026

Instrumenting AI Agents

Seven ways to instrument the same agent doing the same job, measured end to end. What each one emits, how closely it follows the OpenTelemetry GenAI conventions, and what you can actually learn about a run from the telemetry it leaves behind.

The one-sentence version

The libraries agree almost completely on describing a model call and almost not at all on describing an agent — so whether your telemetry can answer "what did this agent do to my database" is decided by which library you installed, not by the specification.

Dash0 · Developer Relations

30 August 2026

CONTENTS

What is in this report

The first half assumes no background in AI or observability. The second half is the measurement. If you only read one thing, read Part 6.

Before you start

3	The vocabulary, in one page	Every term this report uses, defined once.
4	What one span actually looks like	Everything in this report is read out of objects like this one.

Part 1 — The concepts

5	What an AI agent actually is	If you write software but have not built with language models, this is the...
6	What OpenTelemetry records	Three nouns carry the whole model.
7	The GenAI semantic conventions	OpenTelemetry publishes an agreed vocabulary for AI operations.
8	Three families of tool, routinely confused	Most confusion in this space comes from treating three different kinds of...
9	The words the industry actually uses	Three terms get used interchangeably and mean different things.
10	A maturity model to place yourself on	The practice has a rough ladder, and knowing which rung you are on tells...

Part 2 — What we built

11	Seven setups, and why each is in the set	Every setup runs byte-identical work: the same prompt, the same three...
12	How the numbers were produced	One rule governs this report, and it is enforced by a program rather than...

Part 3 — Every setup measured

13	Pydantic AI	Agent framework, self-instrumenting.
14	OpenLIT	Instrumentation library plus platform.
15	CrewAI + OpenLIT	Agent framework, third-party instrumented.
16	Mastra	Agent framework, self-instrumenting (TypeScript).
17	OpenLLMetry	Instrumentation library.
18	LangChain + OpenInference	Agent framework, third-party instrumented.
19	OpenInference	Instrumentation library.
20	OpenTelemetry GenAI	Instrumentation library, the reference.
21	Why OpenInference looks so different	OpenInference is the only project here that wrote its own semantic...
22	Spring AI: everything captured, none of it portable	The second Java framework, and the clearest example in this study of the...
23	Spring AI: recovering the missing field	Spring AI captures the tool result under its own name.
24	Quarkus and LangChain4j: the Java path	Every example so far is Python.
25	Agents you run rather than build	Everything so far is a library you import.
26	kagent: a platform, not a library	The last category is an agent platform running in Kubernetes.
27	Three more agents you install	An SRE agent, a workflow platform and a Kubernetes diagnostic tool.

CONTENTS

What is in this report, continued

Part 4 — Conformance and translation

28	Span names, kinds and status	Three rules from the written specification, checked against every span...
29	Attribute coverage, and what translation fixes	Counting attributes is a weak measure on its own.
30	Translating vocabularies in the collector	One collector component exists specifically because these libraries disagree.
31	What it does for each source	The two built-in sources need opposite amounts of work, and measuring both...
32	Why not just use the transform processor	A fair objection, and one raised during the component's own donation...
33	The same translation, moved into the client	Part 4 has argued that translating between AI vocabularies belongs in the...
34	Two donations, one accepted	Both of the vocabularies the collector translates were offered to...
35	What the acceptance actually promised	The conditions attached to the OpenInference donation are the most useful...
36	The protocols moved to a different foundation	While the GenAI conventions were being written inside OpenTelemetry, the...
37	The working group, and what it is aiming at	The charter is worth reading in full if you work in this area.

Part 5 — What the telemetry says

38	Reading the model's own words	The most useful thing in the whole specification is not about tools.
39	Reconstructing the run	The real test is whether you could reconstruct what the agent did with the...
40	What is actually worth evaluating	“Evaluate your agent” is six different jobs with six different...
41	agent-health, run against a real judge	OpenSearch’s agent-health is the only tool in this study built...
42	Two projects, two philosophies	Both OpenInference and OpenLIT support evaluation.
43	Three places to put the judge	Evaluation is one mechanism — a second model call with a rubric...
44	The second axis: when it runs	Where the judge sits is only half the design.

Part 6 — Comparison

45	Everything measured, side by side	One table.
46	Beyond the seven: everything else measured	The seven Python setups are a controlled comparison: same language, same...
47	Where this work fits at Dash0	This study is a measurement of the ecosystem, not a product pitch.
48	The coding agent is the nearest case	The cheapest place to apply everything in this report is not production.
49	When to use what	The honest summary: no single choice is best, and the right one depends...

Appendices

50	Every GenAI attribute observed, part 1	All 49 distinct gen_ai.* attributes captured across the seven Python...
51	Every GenAI attribute observed, part 2	
52	Where the reasoning goes	Modern models think before answering.

How to read the colours

Throughout, **green** means the information is present under the name the specification defines, so a portable query finds it. **Amber** means the information exists but only under the library’s own private name. **Grey** means it is not there at all. The distance between green and amber is the entire subject of this report.

BEFORE YOU START

The vocabulary, in one page

Every term this report uses, defined once. Nothing here assumes prior knowledge of AI systems or of observability tooling. Skip it if the words are already familiar.

The AI side

Large language model (LLM)

A service you send text to and get text back from. It is stateless: every request carries the whole conversation so far.

Token

The unit models read and write in — roughly three-quarters of a word. Providers bill per token, so token counts are the closest thing to a cost figure in telemetry. A run in this study used 450–2,100 input tokens.

Tool

An ordinary function in your code that you tell the model it may use. The model never runs it. It replies asking you to run it, and your code decides whether to.

Tool call versus tool execution

Two different events, and the distinction drives most of this report. The **call** is the model asking. The **execution** is your code running the function. Instrumentation that watches the model provider sees only the first.

Agent

A loop around the model: call it, run any tool it asks for, feed the result back, repeat until it answers. Nothing more exotic than that.

Reasoning / thinking

An internal step some models perform before answering. It is billed and it is often not returned to you.

LLM-as-a-judge

Using a second model to score the first one's answer against a criterion, such as relevance or harmfulness. It needs the answer text as input.

The observability side

OpenTelemetry

A vendor-neutral standard for producing telemetry, plus the libraries that implement it. Its point is that you instrument once and can send the data anywhere.

Span, trace, attribute

A **span** is one timed operation with a name and a bag of key/value **attributes**. A **trace** is a set of spans sharing an id, forming one unit of work.

Span kind

Whether a span represents calling out to something (`CLIENT`), serving a request (`SERVER`), or work inside your own process (`INTERNAL`). Backends use it to draw service dependency maps.

OTLP

OpenTelemetry Protocol — the wire format telemetry travels in. Every tool in this report speaks it, which is why they can all be pointed at the same place.

Collector

A service that receives telemetry, optionally transforms it, and forwards it on. A **processor** is one transformation step inside it.

Instrumentation

Code that produces telemetry about other code, usually by wrapping a library's functions. An **instrumentation scope** is the name and version it stamps on every span it creates — used in this report to prove nothing was fabricated.

Semantic conventions

The agreed names for attributes. Without them every library invents its own key for "which model ran" and no query works across two services.

Cardinality

How many distinct values a field takes. Span names should be low-cardinality — a class of operation, not one per request — or grouping breaks.

Maturity labels used throughout

OpenTelemetry marks how settled a thing is. **Development** means names may still change — the entire GenAI convention area is here. **Alpha** means usable but unstable, which is the `gen_ai_normalizer` processor's status. **Beta** is the reference GenAI instrumentation. **Stable** means the names will not change, and nothing in this report is stable yet. Read every finding with that in mind.

BEFORE YOU START

What one span actually looks like

Everything in this report is read out of objects like this one. It is a real span from the study, lightly trimmed, with the parts labelled.

```
// the span's identity
name     : "chat claude-opus-5"  ← should be "{operation} {model}"
kind     : CLIENT  ← it calls out over the network
status   : Ok  ← the spec says leave this Unset
traceId  : 4b1c9f...  ← ties spans into one run
duration : 2.4s

// attributes — where everything interesting lives
gen_ai.operation.name     : "chat"
gen_ai.provider.name      : "anthropic"
gen_ai.request.model      : "claude-opus-5"
gen_ai.usage.input_tokens  : 602  // what it cost
gen_ai.usage.output_tokens : 126
gen_ai.response.finish_reasons: ["tool_use"]  // why it stopped
gen_ai.output.messages    : "[{role:assistant, parts:[...]}]"  ← what it said
gen_ai.tool.name          : "list_records"  ← what it wanted to do
gen_ai.tool.call.arguments : "{"plan": "free"}"
gen_ai.tool.call.result    : — absent on six of seven libraries

// who created this span — used to prove it is genuine
scope    : openlit.instrumentation.anthropic
```

Reading it

The top block is the span's **identity**: what it was, what kind of operation, whether it succeeded, and which run it belongs to. Part 4 of this report checks those four fields against the specification, because they are the cheapest thing to get right and five of seven libraries get at least one wrong.

The middle block is **attributes**, and it is where every other finding comes from. Note the shape of the names: `gen_ai.` is the agreed prefix, then a category (`usage`, `request`, `tool`), then the field. A library that writes `llm.model_name` instead of `gen_ai.request.model` is recording the same fact where no standard query will find it.

The greyed line is the single most consequential absence in this study. `gen_ai.tool.call.result` is defined by the specification and emitted by one of the seven libraries tested.

> **Three questions to keep in mind**
>
> For any span you meet: **is the information present at all**, **is it under the name the specification defines**, and **can you tell which run it belongs to**. Those three questions structure everything that follows, and different libraries fail different ones.

PART 1 · CONCEPTS

What an AI agent actually is

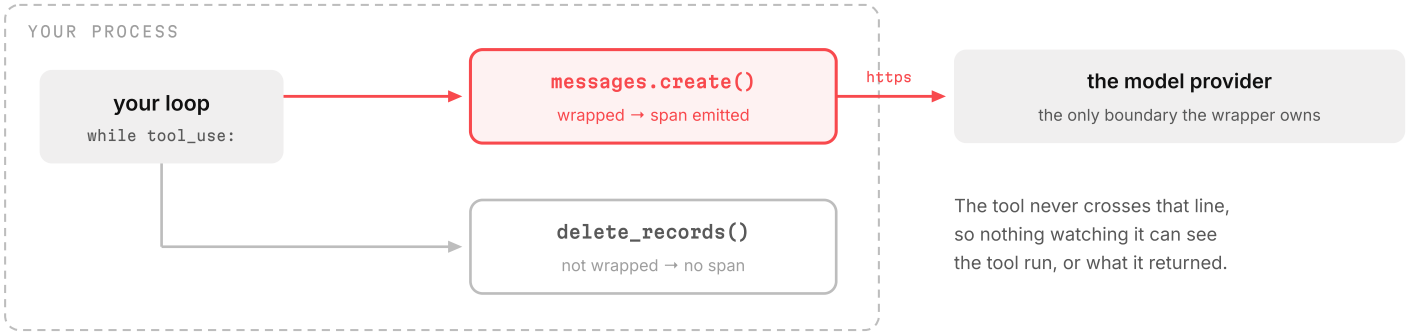
If you write software but have not built with language models, this is the one idea that makes everything else legible.

A model call is a function call. An agent is a loop.

Calling a large language model is an ordinary HTTP request. You send some text and a list of tools it is allowed to use; it sends back either an answer or a request to run one of those tools. That is all a single call does. It cannot run anything itself.

An **agent** is the loop you write around that. Call the model. If it asked for a tool, run the tool, append the result, call the model again. Repeat until it answers instead of asking. In Python it is about fifteen lines:

```
while True:
    response = client.messages.create(model=..., tools=[...], messages=messages)
    if response.stop_reason != "tool_use":
        return response.text
    result = delete_records(**response.tool_input)  # your code runs here
    messages.append(result)
```



Why four of the seven libraries produced no tool span at all: they wrap the provider SDK, and the tool executes entirely outside it.

The commented line is where this report begins. **The model never runs your tool.** It asks; your loop executes. The model provider sees the request and the answer, and never sees what your function did.

Why that matters for observability

Most tools that record AI telemetry work by wrapping the provider's SDK — they patch `messages.create` and record what crosses it. That gives them the prompt, the answer, and the token counts. It cannot give them the tool execution, because the tool never crosses that boundary.

So the same question — *what did this agent actually do?* — is easy or impossible depending on whether something in your process can see the loop. That is the split this report measures, and it explains most of the differences between the seven libraries tested.

The example used throughout

Every measurement in this report comes from one agent given one job: clean up inactive customer records on a free plan, preferring reversible archiving over deletion. It has three tools. `list_records` reads. `archive_records` always fails. `delete_records` works and is destructive. The failure is deliberate: how an agent recovers is where runs differ, and whether the telemetry shows that is the test.

PART 1 · CONCEPTS

What OpenTelemetry records

Three nouns carry the whole model. If you know them, skip to the next page.

Span, trace, attribute

A **span** is one timed operation: it has a name, a start and end time, and a bag of key/value **attributes**. A database query is a span. An HTTP request is a span. A model call is a span.

A **trace** is a set of spans that share an id and form a parent/child tree, so one trace is one end-to-end unit of work. Being able to ask a backend "show me that whole operation" depends entirely on the spans sharing a trace.

Spans also carry two fields that are easy to overlook and matter later: a **kind** (is this a client calling out, a server receiving, or internal work) and a **status** (unset, ok, or error).

Metrics and logs are separate signals

Alongside traces, OpenTelemetry carries **metrics** (numbers aggregated over time, such as token usage) and **logs** (individual records). They travel in separate pipelines. A component that processes traces does not see metrics or logs. That constraint has consequences in Part 5.

The collector

Applications send telemetry to a **collector**: a service that receives, transforms and forwards it. Transformation matters here because one collector component, the `gen_ai_normalizer` processor, rewrites AI attribute names from one vocabulary into another in flight. It is used throughout this report and its limits are measured in Part 4.

> **Why names are the whole problem**
>
> Attributes are free-form key/value pairs. Nothing stops one library writing `llm.model_name` and another writing `gen_ai.request.model` for the same fact. Both are valid OpenTelemetry. Neither is queryable alongside the other. A specification is how an industry agrees on the keys — and, as Part 4 shows, agreeing on keys turns out not to be enough.

PART 1 · CONCEPTS

The GenAI semantic conventions

OpenTelemetry publishes an agreed vocabulary for AI operations. It is young, it is incomplete, and it is the yardstick used throughout this report.

What they define

The conventions live in their own repository, `open-telemetry/semantic-conventions-genai`, and the whole area is still marked **Development** — the least stable maturity level, meaning names can still change. They cover four things:

AREA	EXAMPLES	WHAT IT TELLS YOU
Spans	`gen_ai.operation.name`, `gen_ai.request.model`, `gen_ai.usage.input_tokens`	What ran, on which model, at what token cost
Content	`gen_ai.input.messages`, `gen_ai.output.messages`	The conversation itself, as structured JSON
Tools	`gen_ai.tool.name`, `gen_ai.tool.call.arguments`, `gen_ai.tool.call.result`	Which tool ran, with what, returning what
Metrics & events	`gen_ai.client.token.usage`, `gen_ai.evaluation.result`	Aggregates, and evaluation outcomes

Three rules that decide most of Part 4

- **Span name** should be `{operation} {model}` — for example `chat claude-opus-5`, or `execute_tool delete_records`.
- **Span kind** should be `CLIENT` for a model call, because it crosses a process boundary. `INTERNAL` is reserved for models running in your own process.
- **Span status** should be left `Unset` by instrumentation on success. The specification is explicit: libraries "SHOULD NOT set the status code to Ok". That value is reserved for an application developer asserting success.

Content is opt-in, by design

Prompts and responses may contain personal data, credentials or customer content, so the conventions mark `gen_ai.input.messages` and `gen_ai.output.messages` **opt-in**: instrumentation should not capture them by default. That is the correct default for a specification serving everyone. As Part 5 shows, exactly one library in this test obeys it — and obeying it costs you almost everything you would want during an incident.

PART 1 · CONCEPTS

Three families of tool, routinely confused

Most confusion in this space comes from treating three different kinds of product as one category. They answer different questions and you may need one of each.

01 **Agent frameworks — you build *in* them**

Pydantic AI, LangChain, CrewAI, LangGraph, Mastra. They own the agent loop: you declare tools and a prompt, and the framework decides when to call the model and when to run a tool. Because they own the loop, they can see the tool execute.

Chosen for: how quickly you can express the agent you want.

02 **Instrumentation libraries — you attach them**

OpenInference (Arize), OpenLLMetry (Traceloop), OpenLIT, and OpenTelemetry's own `opentelemetry-python-genai`. They patch something — a provider SDK, or a framework — and emit telemetry about it. They do not change how your agent behaves.

Chosen for: what they record, and under which names.

03 **Backends and evaluation platforms — you send data to them**

Dash0, Langfuse, Arize Phoenix, LangSmith, the OpenLIT platform. They store, display and sometimes score telemetry. Some also ship their own SDK, which is why they are so often mistaken for the second category.

Chosen for: what you can ask of the data once it has arrived.

The pairing is what matters

A framework and an instrumentation library are chosen separately, by different people, for different reasons — and the combination decides your telemetry. Pydantic AI instruments itself. LangChain does not, so you pair it with one of the libraries above. That pairing, not the framework alone, determines whether you can answer a question later.

PART 1 · CONCEPTS

The words the industry actually uses

Three terms get used interchangeably and mean different things. Getting them straight is worth a page, because each one names a different layer of the problem and this report is mostly about the second.

01 **LLM observability — what did the *model* do?**

Visibility into the language-model calls themselves: prompts, completions, token counts, latencies, model versions, error rates. It is the foundation layer, and the closest analogue to APM for a traditional service.

Where this report lands: largely solved. Every library measured describes a model call competently, and five attributes are universal across all seven.

02 **Agent observability — what did the *agent* do, and why?**

Visibility one level up: goals, plans, tool calls, intermediate decisions, retries, escalations and outcomes. **The unit of analysis is the run** — a goal entered, a plan formed, steps executed, an outcome reached. Each run is a trace, each step a span, each tool call a child span.

Where this report lands: this is the broken layer. Four of the seven setups do not produce a run at all, because provider-SDK instrumentation emits one disconnected trace per model call. And the field that says what a tool returned is absent from six of seven.

03 **Agentic observability — two meanings, routinely confused**

The first is agents as the thing **observed**, which is the sense above. The second flips it: agents as the thing **doing the observing** — an AI reading your telemetry, correlating signals and proposing fixes.

Both are real and they depend on each other. An agent that investigates your systems is only as good as the telemetry it can read, which is decided by everything in the first half of this report.

The three layers of signal

Within LLM observability, a complete picture spans three kinds of signal, and most teams underinvest in the last two:

LAYER	WHAT IT COVERS	WHAT THIS STUDY MEASURED
Operational	Latency, throughput, provider error rates, token usage. Tokens matter most because they map straight to cost.	Well covered. All seven emit token counts; two emit a dollar figure.
Semantic	Correctness, relevance, faithfulness, safety. Cannot come from infrastructure — it requires evaluation.	Possible for four of seven, since only those carry the model's answer text under a spec name.
Agentic	**The multi-step execution tree: which tool ran with which arguments, which branch went sideways, where tokens pile up across turns.**	**Weakest by far. One library and one coding agent give a complete trajectory.**

Why traditional monitoring is not enough, in three sentences

Non-determinism: the same elevated latency can mean a long reasoning chain, an over-eager retrieval step, or a tool timing out three hops in — same symptom, unrelated causes. **Correctness lives outside the status code:** a response that invents a policy returns HTTP 200 every time. **Cost is driven by tokens, not requests:** an application can be fast, error-free and quietly expensive all at once.

A maturity model to place yourself on

The practice has a rough ladder, and knowing which rung you are on tells you what to build next better than any feature list.

LEVEL	WHAT YOU HAVE	WHAT IT ANSWERS	WHAT THIS STUDY FOUND
L0	Prompt logs	What was asked and answered, with a timestamp	—
L1	Model telemetry	Model, latency, tokens, errors, cost	All seven setups reach this
L2	**Distributed AI traces**	**Retrieval, tools, orchestration, trace propagation, sessions**	**Three of seven. The provider-hook libraries cannot, structurally**
L3	Quality observability	Automated evaluation, human labels, user feedback, outcomes	Possible for four; only one project writes the spec's evaluation event
L4	Continuous evaluation	Production evaluation, datasets, experiments, CI	Not reached by anything measured
L5	Governed agent observability	Security signals, identity, permissions, memory provenance, trajectories, MCP, privacy controls	Not reached

The honest reading of this report against that ladder: **the ecosystem is solidly at L1, patchy at L2, and only beginning at L3.** Every library describes a model call. Fewer than half produce a connected trace of a run. One writes an evaluation result in the shape the conventions define.

Anti-patterns worth naming

- **"We log every prompt and response."** That is logging, not observability. It has no causal structure.
- **"We have a gateway, so we can observe the AI system."** A gateway cannot see retrieval, application routing, memory, internal tools or agent state.
- **"The LLM judge says we are at 92%."** A number without a rubric, an evaluator version, a sample definition and human calibration is weak evidence. This study produced a metric that read **1.00** for a run that recorded nothing at all.
- **"The final answer looked correct."** An agent can produce a plausible answer after an incorrect trajectory — which is precisely why the tool result matters.
- **"We can reproduce it from the prompt."** Not without the model, prompt version, tools, retrieval index, flags, memory and evaluator.

The privacy paradox

The data most useful for debugging is often the data most dangerous to retain. That is the real reason the conventions mark content opt-in, and the real reason the tool result is the hardest field to get: the half you would least want to log by default is the half that names what your agent actually did. Treat the telemetry pipeline as a sensitive data system — redact before export, separate fields by sensitivity, prefer identifiers over content, and set retention per data class rather than globally.

Seven setups, and why each is in the set

Every setup runs byte-identical work: the same prompt, the same three tools, the same model. Only the instrumentation differs, so any difference in the telemetry belongs to the library rather than to the code.

SETUP	WHO OWNS THE LOOP	INSTRUMENTATION	WHY IT IS IN THE SET
pydantic-ai	**the framework**	**native, built in**	**The only one that instruments itself and claims full convention conformance. The control for "what good looks like".**
openlit	your own code	OpenLIT	Claims to emit specification keys directly. Also emits metrics and a spec-defined event, so it tests all three signals.
crewai	the framework	OpenLIT	A framework instrumented by a third party rather than by itself. Tests whether framework plus external library is enough.
openllmetry	your own code	OpenLLMetry	Widely deployed, and one of two source vocabularies the collector's normalizer knows how to translate.
langchain	the framework	OpenInference	The most common production pairing. LangChain has no OpenTelemetry of its own.
openinference	your own code	OpenInference	The other vocabulary the normalizer translates, and the one needing the most translation.
otel-genai	your own code	OpenTelemetry's own	The reference implementation, written by the people who own the specification. The baseline everything else is read against.

Each setup is a self-contained example

Every one is its own directory with its own dependency list and its own isolated Python environment. This is not tidiness: these libraries pin conflicting versions of the OpenTelemetry SDK, and installing them together means measuring whichever one won the install rather than the one you meant to test.

What they all send to

One OpenTelemetry collector, contrib distribution 0.159.0, running the `gen_ai_normalizer` processor with OpenInference and OpenLLMetry as translation sources. Traces pass through the normalizer; metrics and logs deliberately bypass it, because the processor is traces-only and making that visible is part of the measurement. Everything is written to local files and read back by the analysis scripts.

PART 2 · METHOD

How the numbers were produced

One rule governs this report, and it is enforced by a program rather than by good intentions.

Rule zero: no fabricated telemetry

No attribute in this report was written by the test harness. Every one was emitted by a real instrumentation library, observing a real agent, calling a real model, executing a real tool.

How it is enforced

Every OpenTelemetry span carries an **instrumentation scope** naming what created it. A real library sets its own name and version; a hand-opened tracer would name the harness. So the rule is checkable rather than promised. A gate reads the scope of every span before any analysis runs, and refuses to produce a report if any span could have been written by the harness. Nine scopes passed in the run behind this report:

INSTRUMENTATION SCOPE	VERSION
`pydantic-ai`	2.36.0
`opentelemetry.util.genai.handler`	1.1b0
`openinference.instrumentation.anthropic`	2.1.1
`openinference.instrumentation.langchain`	0.1.73
`opentelemetry.instrumentation.anthropic`	0.62.3
`openlit.instrumentation.anthropic`	—
`openlit.instrumentation.crewai`	—
`opentelemetry.instrumentation.httpx`	0.63b1 / 0.65b0

The consequence to accept: where a library genuinely emits nothing, this report says nothing. An empty cell is a finding. Filling one in by hand would destroy the only thing the harness is for.

PART 3 · PROFILE

Pydantic AI

Agent framework, self-instrumenting

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
27	5	2032	0
distinct `gen_ai.*` keys	14 over 3 runs	chars an evaluator can read	no events

What it is, and who uses it

From the team behind Pydantic, the validation library underneath most of the Python AI ecosystem — including the SDKs of several other tools in this report. That lineage matters: they were already the layer everyone depended on before they shipped an agent framework.

The project's goal

Bring Pydantic's validation-first, type-checked approach to agent building. The pitch is that an agent is ordinary typed Python: tools are functions with type hints, outputs are validated models, and your editor and type checker work throughout.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`pydantic/pydantic-ai`	19.6k	2.6k	475	11.6M	June 2024	MIT

What it does in this study

A typed agent framework from the Pydantic team. You declare tools as ordinary Python functions with type hints; it validates arguments and owns the loop.

How you set it up

`Agent.instrument_all()` plus a standard OpenTelemetry tracer provider. No instrumentation package to install — it emits OpenTelemetry directly.

Content switch

`InstrumentationSettings(include_content=True, version=5)`. A named parameter, and the only library here that lets you choose which generation of the conventions to emit (2 through 6).

What we observed

Four spans per run: `invoke_agent agent`, `chat claude-opus-5`, `execute_tool list_records`, and a second chat. Span names, kinds and status all match the specification exactly.

SIGNAL	MEASURED
Span names	`chat claude-opus-5`, `execute_tool archive_records`, `execute_tool list_records`, `invoke_agent agent`
Span kinds / status	CLIENT, INTERNAL · status Unset
Finish reason values	`stop`, `tool_call`
Tool coverage	NAME ARGS RESULT
Output message parts	`text`, `thinking`
Metrics	—

Stands out

The only one of the seven Python setups that emits `gen_ai.tool.call.result` at its defaults. Also captures the model's reasoning as a `thinking` part inside the output messages. Two qualifiers: Mastra does the same in TypeScript, and the Goose coding agent does it once its content switch is turned on.

Falls short

Emits no metrics at all, so token cost and latency have to be derived from spans. Ties you to the Pydantic AI framework.

Verdict

The reference for what conformant agent telemetry looks like. If you are starting a Python agent today and observability matters, this is the shortest path to a complete answer.

PART 3 · PROFILE

OpenLIT

Instrumentation library plus platform

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
28	2	1806	5
distinct `gen_ai.*` keys	12 over 6 runs	chars an evaluator can read	and the spec event

What it is, and who uses it

The smallest project measured here by stars, and the most complete by telemetry. It positions itself as OpenTelemetry-native rather than OpenTelemetry-compatible, and the measurements bear that out: it is the only library emitting spans, metrics and the specification's own event.

The project's goal

Be the OpenTelemetry-native option for AI engineering: one library that instruments models, frameworks, vector databases and GPUs, paired with a self-hostable platform for dashboards, evaluations and guardrails.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`openlit/openlit`	2.7k	366	104	402k	January 2024	Apache-2.0

What it does in this study

A broad instrumentation library covering many providers and frameworks, paired with an optional self-hosted platform for dashboards and evaluation.

How you set it up

One call: `openlit.init(otlp_endpoint=...)`. It auto-instruments whatever it finds installed, including the HTTP client underneath.

Content switch

`capture_message_content=True` — a single named parameter on init. The most discoverable content switch of anything tested.

What we observed

Two spans per run named `chat claude-opus-5`, correctly following the naming convention, plus two `POST` spans it adds by auto-instrumenting httpx. 28 convention attributes, the second richest.

SIGNAL	MEASURED
Span names	`POST`, `chat claude-opus-5`
Span kinds / status	CLIENT · status Ok, Unset
Finish reason values	`end_turn`, `tool_use`
Tool coverage	NAME ARGS NO RESULT
Output message parts	`text`
Metrics	client.operation.duration, client.token.usage, server.request.duration, server.time_to_first_token, usage.cost

Stands out

The only library emitting all three signals: spans, five GenAI metrics including `gen_ai.usage.cost` in dollars, and the specification's own `gen_ai.client.inference.operation.details` event as a log record.

Falls short

Sets span status to `Ok`, which the trace specification says instrumentation should not do. Pins `anthropic<1.0.0`, so it cannot instrument the current Anthropic SDK. No tool result.

Verdict

The most complete telemetry of any non-native option, and the only one that gives you cost. Choose it when you want breadth across providers and a dollar figure without building one.

PART 3 · PROFILE

CrewAI + OpenLIT

Agent framework, third-party instrumented

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
35	25	1866	5
distinct `gen_ai.*` keys	74 over 3 runs	chars an evaluator can read	and the spec event

What it is, and who uses it

The second most popular project in this report and the only one built around multi-agent collaboration rather than a single agent with tools. That design shows up directly in its telemetry, which is an order of magnitude more verbose than anything else measured.

The project's goal

Orchestrate multiple role-playing agents that collaborate on a task. You describe each agent by role, goal and backstory; the framework handles delegation between them.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`crewAIInc/crewAI`	57.8k	8.3k	301	28.4M	October 2023	MIT

What it does in this study

A role-based multi-agent framework: you describe agents by role, goal and backstory, and they coordinate. Instrumented here by OpenLIT rather than by itself.

How you set it up

Define an `Agent` with a role, goal, backstory and tools, then `agent.kickoff(prompt)`. Telemetry comes from `openlit.init()` as above.

Content switch

Inherits OpenLIT's `capture_message_content`. CrewAI has no telemetry switch of its own that reaches OpenTelemetry.

What we observed

By far the most verbose: 52 spans across two runs, against four per run for Pydantic AI. Most are named `invoke_agent` `check_max_iterations`, `invoke_agent finalize` and similar — internal state-machine nodes, not agent invocations.

SIGNAL	MEASURED
Span names	`POST`, `chat claude-opus-5`, `invoke_agent Platform maintenance engineer`, `invoke_agent call_llm_native_tools`, `invoke_agent check_max_iterations`
Span kinds / status	CLIENT, INTERNAL · status Ok, Unset
Finish reason values	`end_turn`, `stop`, `tool_use`
Tool coverage	NAME ARGS NO RESULT
Output message parts	`text`
Metrics	client.operation.duration, client.token.usage, server.request.duration, server.time_to_first_token, usage.cost

Stands out

35 convention attributes, the highest count in the set, plus metrics and the spec event inherited from OpenLIT.

Falls short

The naming is right in form and wrong in meaning, which is worse than an obviously wrong name because it validates cleanly. And despite published claims to the contrary, this pairing does **not** emit `gen_ai.tool.call.result`.

Verdict

Choose CrewAI for multi-agent orchestration, not for telemetry. Expect to filter aggressively: most of its spans describe the framework's internals rather than your agent's work.

PART 3 · PROFILE

Mastra

Agent framework, self-instrumenting (TypeScript)

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
23	17	1965	0
distinct `gen_ai.*` keys	52 over 3 runs	chars an evaluator can read	and 1 unnamed log record

What it is, and who uses it

The only TypeScript framework measured here, and the only project in the study whose exporter documentation names the version of the semantic conventions it targets. Downloads are for `@mastra/core` on npm rather than PyPI. It is also the only framework that ships a first-party Dash0 provider preset.

The project's goal

Be the TypeScript agent framework for people already building on the Vercel AI SDK: agents, workflows, memory, evals and deployment, with observability treated as a first-class part of the framework rather than an add-on.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	NPM / MONTH	SINCE	LICENCE
`mastra-ai/mastra`	27.6k	2.7k	465	5.9M	August 2024	Elastic-2.0

What it does in this study

A TypeScript agent framework built over the Vercel AI SDK. You declare tools with `createTool` and zod schemas; it owns the loop and the observability pipeline.

How you set it up

`new Observability({configs: {otel: {exporters: [new OtelExporter({provider: {custom: {endpoint, protocol}}})]}}})` passed to the Mastra constructor, plus the matching OTLP exporter package. A documented `custom` provider points at any collector; no Mastra account.

Content switch

None needed. Content is captured by default, and the `signals` option toggles traces and logs rather than content.

What we observed

Around 17 spans per run. Three are exactly what the specification asks for: `chat claude-opus-5` as `CLIENT`, `execute_tool listRecords`, and `invoke_agent Maintenance Agent`. The rest are Mastra's own `model_chunk`, `model_step`, `model_inference` and `processor_run` spans.

SIGNAL	MEASURED
Span names	`chat claude-opus-5`, `execute_tool archiveRecords`, `execute_tool listRecords`, `invoke_agent Maintenance Agent`, `model_chunk Maintenance Agent`
Span kinds / status	CLIENT, INTERNAL · status Ok, Unset
Finish reason values	`["stop"]`
Tool coverage	NAME ARGS RESULT
Output message parts	`text`
Metrics	—

Stands out

23 convention attributes, and the **second library in the entire study to emit** `gen_ai.tool.call.arguments` **and** `gen_ai.tool.call.result` — the field this whole report is about. It also emits `gen_ai.system_instructions`, `gen_ai.tool.definitions`, agent identity as `gen_ai.agent.id` and `.name`, and `gen_ai.usage.reasoning_tokens`, which nothing else here reports.

Falls short

Two layers, and only one is portable: internally Mastra records `mastra.*` attributes and the exporter translates on the way out, so a bug in the exporter loses everything. Roughly four in five spans describe streaming internals rather than agent work. No metrics.

Verdict

The strongest TypeScript result by a wide margin, and the clearest demonstration that a framework can be conformant without giving up its own model. If you are building an agent in TypeScript and observability matters, start here.

PART 3 · PROFILE

OpenLLMetry

Instrumentation library

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
15	1	1986	3
distinct `gen_ai.*` keys	6 over 6 runs	chars an evaluator can read	no events

What it is, and who uses it

One of the earliest entrants, and the most widely adopted dedicated GenAI instrumentation by downloads. Its interoperability pitch is the reason it converged on the conventions early, which the measurements confirm. Its donation to OpenTelemetry is covered in Part 4.

The project's goal

Provide OpenTelemetry-based observability for LLM applications across more than 40 integrations — foundation models, frameworks and vector databases — so telemetry can be sent to any OTLP backend rather than a single vendor.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`traceloop/openllmetry`	7.4k	1.1k	108	2.0M	September 2023	Apache-2.0

What it does in this study

Traceloop's instrumentation suite, covering foundation models, frameworks and vector databases. Widely deployed and one of two vocabularies the collector normalizer translates.

How you set it up

`Traceloop.init(api_endpoint=...)`. One call, and `disable_batch=True` for short-lived scripts.

Content switch

`TRACELOOP_TRACE_CONTENT`, an environment variable defaulting to true. Also exposes `use_legacy_attributes`, which selects between legacy and current attribute names.

What we observed

The leanest output in the set: one span per model call, 15 attributes, **every one of them already a convention attribute**. The normalizer had nothing to translate.

SIGNAL	MEASURED
Span names	`anthropic.chat`
Span kinds / status	CLIENT · status Ok
Finish reason values	`stop`, `tool_call`
Tool coverage	NO NAME NO ARGS NO RESULT
Output message parts	`reasoning`, `text`
Metrics	client.generation.choices, client.operation.duration, client.token.usage

Stands out

Full convergence on the model call, confirmed by measurement. It is also one of only two libraries that records the model's reasoning, as a `reasoning` part inside the output messages.

Falls short

No tool attributes whatsoever — not even the tool name. Span named `anthropic.chat` rather than the specified `chat {model}`. Sets status `Ok`. The published package fails to declare a dependency it imports, so a clean install cannot be imported.

Verdict

Excellent for model-call observability and useless for agent-loop observability. A good choice if your question is about cost, latency and output quality rather than about actions taken.

PART 3 · PROFILE

LangChain + OpenInference

Agent framework, third-party instrumented

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
9	10	0	0
distinct `gen_ai.*` keys	29 over 3 runs	chars an evaluator can read	no events

What it is, and who uses it

By a wide margin the largest project in this study: more stars than the other six combined and roughly 239 million downloads a month. It defined much of the vocabulary the field uses. Observability is not part of its core offering — natively it reports to LangSmith, its commercial platform — which is why it is paired here with a third-party instrumentation.

The project's goal

Be the general-purpose framework for building on language models — chains, agents, retrieval, memory, and integrations with essentially every provider and vector store.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`langchain-ai/langchain`	145k	24.3k	>3k	239M	October 2022	MIT

What it does in this study

The most widely used agent framework. It has no OpenTelemetry of its own — natively it reports to LangSmith — so it is paired here with OpenInference.

How you set it up

`create_agent(model=..., tools=[...])` then `agent.invoke(...)`. Telemetry via `LangChainInstrumentor().instrument(...)`.

Content switch

OpenInference's nine separate `hide_*` flags, all defaulting to capture. There is no single content switch.

What we observed

Seven to eleven spans per run, including a real `delete_records` tool span — framework instrumentation does see the tool execute. But spans are named after implementation details: `ChatAnthropic`, `model`, `tools`, `LangGraph`.

SIGNAL	MEASURED
Span names	`ChatAnthropic`, `LangGraph`, `archive_records`, `list_records`, `model`
Span kinds / status	INTERNAL · status Ok
Finish reason values	`end_turn`, `tool_use`
Tool coverage	NAME NO ARGS NO RESULT
Output message parts	`tool_call`
Metrics	—

Stands out

It produces a real tool span with `gen_ai.tool.name`, so you can tell which tool ran and in what order — and it captures more raw content than any other setup measured.

Falls short

Everything valuable is under private names. The normalized `gen_ai.output.messages` holds a tool call and **zero characters of model text**, while `output.value` on the same spans holds 12,496 characters — the most content of anything tested. All spans marked `INTERNAL` and status `Ok`.

Verdict

The most popular framework produces the most content and the least portable content. Everything is there if you query OpenInference's vocabulary. Nothing useful is there if you query the specification's.

PART 3 · PROFILE

OpenInference

Instrumentation library

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
7	1	0	0
distinct `gen_ai.*` keys	7 over 7 runs	chars an evaluator can read	no events

What it is, and who uses it

From Arize, the company behind the Phoenix observability tool. Fewer stars than OpenLLMetry but more downloads on the instrumentation packages, because it is what Phoenix installs. Notably, it is the project whose code grant OpenTelemetry accepted in June 2026 — covered in Part 4.

The project's goal

Supply OpenTelemetry instrumentation for AI observability across Python, JavaScript, Java and Go, alongside its own semantic conventions for describing LLM and agent operations.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`Arize-ai/openinference`	1.2k	302	131	603k	December 2023	Apache-2.0

What it does in this study

Arize's instrumentation suite, donated to OpenTelemetry in June 2026. Uses its own `llm.*` vocabulary throughout and relies on translation to reach the conventions.

How you set it up

Build a tracer provider, then `AnthropicInstrumentor().instrument(tracer_provider=...)`. Four lines, closely following its quickstart.

Content switch

Nine `hide_*` flags — `hide_inputs`, `hide_outputs`, `hide_llm_tools` and six more — all defaulting to False, meaning capture everything.

What we observed

The most verbose source data and the least standard: 36 to 42 attributes per span, **none** of them convention attributes. All seven `gen_ai.*` attributes on those spans were added by the collector. It records 2,454 characters of model text and a 4,969-character copy of the entire raw API response — under its own names.

SIGNAL	MEASURED
Span names	`messages.create`
Span kinds / status	INTERNAL · status Ok
Finish reason values	`end_turn`, `tool_use`
Tool coverage	NO NAME NO ARGS NO RESULT
Output message parts	`tool_call`
Metrics	—

Stands out

It captures more content than any other library here: the answer text, the reasoning block, the reasoning signature, and the complete raw response in `output.value`. Nothing is lost at the library.

Falls short

None of that content survives normalization. The text and reasoning live in an indexed array form the processor documents that it does not reconstruct, so the normalized `gen_ai.output.messages` arrives holding a tool call and zero characters of text. Span named `messages.create` after the SDK method; marked `INTERNAL` for a remote HTTPS call; status `Ok`. No metrics.

Verdict

The richest capture and the least portable. Everything you would want is present; almost none of it is under a name the specification defines. Query it in OpenInference's own vocabulary and it is excellent; query it as OpenTelemetry and it is nearly empty.

PART 3 · PROFILE

OpenTelemetry GenAI

Instrumentation library, the reference

CONVENTION ATTRIBUTES	SPANS PER RUN	MODEL TEXT CAPTURED	GENAI METRICS
11	1	0	0
distinct `gen_ai.*` keys	6 over 6 runs	chars an evaluator can read	no events

What it is, and who uses it

Three months old at the time of measurement and the smallest by every adoption measure — 35 stars and roughly 5,400 downloads a month, against 239 million for LangChain. It is included not because anyone runs it at scale yet, but because it defines what conformance means: it is the yardstick, not a competitor.

The project's goal

Provide the reference implementation of the GenAI semantic conventions, maintained by the same project that writes them. It is where OpenInference's donated instrumentations are being incorporated.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	PYPI / MONTH	SINCE	LICENCE
`open-telemetry/opentelemetry-python-genai`	35	54	172	5.4k	May 2026	Apache-2.0

What it does in this study

OpenTelemetry's own instrumentation, in a dedicated repository separate from contrib. Written by the people who own the conventions, and the implementation the project's conformance suite measures.

How you set it up

`AnthropicInstrumentor().instrument(tracer_provider=...)` from `opentelemetry.instrumentation.genai.anthropic`. Currently beta, version 1.1b1.

Content switch

`OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT`, plus a family of environment variables for event emission and content offloading. Environment variables only, no code parameter.

What we observed

One span per model call named `chat claude-opus-5`, kind `CLIENT`, status left `Unset`. Twelve attributes, eleven of them convention attributes. Fully conformant on every span-level rule tested.

SIGNAL	MEASURED
Span names	`chat claude-opus-5`
Span kinds / status	CLIENT · status Unset
Finish reason values	`stop`, `tool_calls`
Tool coverage	NO NAME NO ARGS NO RESULT
Output message parts	—
Metrics	—

Stands out

Together with Pydantic AI, the only library that gets span name, kind and status all correct. It is the yardstick.

Falls short

No message content, no tool name, no arguments, no result, no metrics. It follows the specification's opt-in default precisely, and out of the box it cannot tell you what your agent did or said.

Verdict

The most conformant and the least informative. It is the honest baseline: this is what the specification alone gets you, and it is why every other library in this list overrides the default.

Why OpenInference looks so different

OpenInference is the only project here that wrote its own semantic conventions rather than adopting OpenTelemetry's. Understanding why explains its unusual telemetry, the existence of the collector's normalizer, and one of the more interesting facts in this report.

It is positioned as complementary, not competing

The project describes itself as "a set of conventions and plugins that is complimentary to OpenTelemetry" — transport-agnostic, usable with any OpenTelemetry-compatible backend, and natively supported by Arize's own tools, Phoenix and Arize AX. It ships roughly 40 instrumentations in Python, 10 in JavaScript, and four each in Java and Go. It was never trying to be the OpenTelemetry conventions; it was trying to describe LLM applications while those conventions did not yet exist.

Difference one: ten span kinds instead of operation names

Where the OpenTelemetry conventions use `gen_ai.operation.name` with a handful of values, OpenInference makes the kind of work a first-class attribute, `openinference.span.kind`, with ten values:

SPAN KIND	WHAT IT MARKS	NEAREST OPENTELEMETRY EQUIVALENT
`LLM`	A model call	`chat`
`TOOL`	A tool invocation	`execute_tool`
`AGENT`	Reasoning combining calls and tools	`invoke_agent`
`CHAIN`	A link between application steps	`invoke_agent` (folded)
`RETRIEVER`	Fetching from a vector store or database	`retrieval`
`RERANKER`	Reordering retrieved documents	`retrieval` (folded)
`EMBEDDING`	Turning text into vectors	`embeddings`
`PROMPT`	Rendering a prompt template	`text_completion`
`GUARDRAIL`	**Content filtering, jailbreak protection**	**none yet**
`EVALUATOR`	**Scoring a model output**	**none yet**

The two highlighted rows connect to Part 4

When the GenAI SIG accepted Arize's code grant in June 2026, the conditions committed maintainers to "work towards covering convention gaps such as supporting additional operation and event types — **guardrails, reranking, evaluations, handoffs**". Three of those four are span kinds OpenInference already defines. The project OpenTelemetry accepted code from had, in that respect, been ahead of the conventions for two years.

Difference two: flattened attributes instead of structured JSON

A conversation in OpenInference is not one JSON blob. It is spread across numbered keys:

```
llm.input_messages.0.message.role    : "user"
llm.input_messages.0.message.content : "Clean up the free plan..."
llm.output_messages.0.message.contents.1.message_content.text : "I'll start by..."
```

This was a deliberate and, at the time, correct decision. OpenTelemetry span attributes were traditionally flat key/value pairs, so a nested structure had nowhere to live. Flattening let backends index and filter individual message fields without parsing JSON. The OpenTelemetry conventions later went the other way, defining `gen_ai.input.messages` as a single structured value.

That divergence is the reason the collector's normalizer needs a message-reconstruction step at all, and it is also the reason some content does not survive translation: the processor rebuilds the flat `message.content` form but documents that it does not rebuild the indexed `message.contents.N` array, which is where the answer text and reasoning actually sit.

Difference three: a generic value container with a declared format

Alongside the typed attributes, every span carries `input.value` and `output.value` — the raw payload as a string — with `input.mime_type` and `output.mime_type` declaring how to read it. That is why an OpenInference span in this study carries a 12,496-character copy of the entire API response. Nothing is lost at the library. It is simply stored in a container the conventions have no equivalent for.

What the donation did and did not transfer

Arize's June 2026 code grant covered **the instrumentation code only** — explicitly not the OpenInference specification or its semantic conventions. So the vocabulary described on this page remains Arize's, continues to evolve independently, and will keep needing translation. What OpenTelemetry gained was coverage across four languages, not a shared dialect.

PART 3 · PROFILE

Spring AI: everything captured, none of it portable

The second Java framework, and the clearest example in this study of the distinction the whole report turns on. It records the tool result. A standard query will not find it.

REPOSITORY	STARS	FORKS	CONTRIBUTORS	SINCE	LICENCE
`spring-projects/spring-ai`	9.4k	2.9k	407	June 2023	Apache-2.0

What it is for

"An Application Framework for AI Engineering" from the Spring team. Its distinguishing choice for this study is the observability path: rather than shipping an AI-specific instrumentation library, it emits **Micrometer observations** and lets `micrometer-tracing-bridge-otel` convert them into OpenTelemetry spans. No AI instrumentation package is installed at all.

What we observed

Ten spans for one run, and the naming is better than most: `chat claude-sonnet-4-6` follows the specification's `{operation} {model}` form exactly. Tool spans are named `tool_call list-records`, which uses Spring's own `tool_call` kind rather than the spec's `execute_tool`.

On the model call — 11 `gen_ai.*`

`operation.name`, `request.model`, `request.max_tokens`, `request.temperature`, `response.model`, `response.id`, `response.finish_reasons`, `usage.input_tokens`, `usage.output_tokens`, `usage.total_tokens`, `gen_ai.system`.

On the tool call — 0 `gen_ai.*`

`spring.ai.tool.definition.name`, `.description`, `.schema`, `spring.ai.tool.call.arguments`, and `spring.ai.tool.call.result`. Everything the forensic question needs, under a namespace only Spring AI uses.

The purest case of captured-but-not-portable in the study

Spring AI records the tool's arguments *and* its result, with a schema and a description, behind a single well-named switch. On the measure that matters — can you reconstruct what the agent did — it is among the best here. On the measure the conventions care about, it scores zero for tools: no `gen_ai.tool.name`, no `gen_ai.tool.call.result`. This is the amber column of this report's colour scheme, in one framework.

One switch for both halves

```
spring.ai.chat.observations.log-prompt       = true
spring.ai.chat.observations.log-completion   = true
spring.ai.tools.observations.include-content = true  ← arguments AND result
```

All default to **false**, which is what the conventions ask for and which almost nothing else in this study manages. One property covers tool arguments and results together, and the documentation carries an explicit warning that enabling it risks exposing sensitive information. That is the "discoverable switch" argument satisfied, with the correct default.

It cannot call the current model either

Spring AI 1.0.0 always sends `temperature`. Claude Opus 5 removed the sampling parameters and returns `400 "temperature is deprecated for this model"`. Setting the property to an empty value does not suppress it. **Both Java frameworks measured fail the same way on the same model** — LangChain4j on `top_k`, Spring AI on `temperature` — and both must fall back to an older model. This example runs Sonnet 4.6.

PART 3 · PROFILE

Spring AI: recovering the missing field

Spring AI captures the tool result under its own name. That turns out to be enough — the field this report keeps finding missing is recoverable without touching the application.

The fix is four lines of collector config, and it works

Spring AI is not a built-in source for `gen_ai_normalizer`, but it is the ideal candidate for a user-defined one: the attributes exist, they are stable, and the mapping is one-to-one. This was not left as a suggestion — it was added to the collector and the example re-run:

```
- name: spring_ai
  mappings:
    spring.ai.tool.definition.name: gen_ai.tool.name
    spring.ai.tool.definition.description: gen_ai.tool.description
    spring.ai.tool.call.arguments: gen_ai.tool.call.arguments
    spring.ai.tool.call.result: gen_ai.tool.call.result
```

Measured after adding that source

ATTRIBUTE	BEFORE	AFTER
`gen_ai.tool.name`	—	present
`gen_ai.tool.description`	—	present
`gen_ai.tool.call.arguments`	—	present
`gen_ai.tool.call.result`	—	**present**

The `spring.ai.*` originals survive alongside them, because `remove_originals` defaults to false.

That last row is the one this whole report has been chasing, and for Spring AI it turns out to be a configuration change — not a code change, not a specification gap, and not a request to a maintainer. **The data was already there.**

It is worth contrasting with OpenInference, where the same trick fails. Its result lives in `output.value`, a generic container also used for the model's own text, so a blind mapping would sometimes label a completion as a tool result. Spring AI named the field precisely, and precise naming is exactly what makes a private vocabulary translatable. **A library that invents its own names is recoverable; one that overloads a generic container is not.**

PART 3 · PROFILE

Quarkus and LangChain4j: the Java path

Every example so far is Python. The conventions are language-agnostic, so the obvious question is whether they hold up somewhere else. This is the same three-tool workload on Quarkus with LangChain4j.

CONVENTION ATTRIBUTES	TOOL RESULT	SPANS PER RUN	DEPRECATED ATTRIBUTES
17	yes	9	2
the second highest measured	under the spec name	incl. one span per tool	removed from the registry

What it emits

Tool spans named after the Java method (`langchain4j.tools.listRecords`, `langchain4j.tools.archiveRecords`), a model span named `completion claude-sonnet-4-6` which follows the specification's `{operation} {model}` shape, and an agent span. Seventeen convention attributes, including the full tool triple — `gen_ai.tool.name`, `gen_ai.tool.call.arguments` and `gen_ai.tool.call.result` — plus `gen_ai.tool.type`, which nothing else in this study emits.

A third implementation that carries the tool result

Pydantic AI in Python, Goose in Rust with its content switch on, and now LangChain4j in Java. The field is not exotic and not blocked. It is emitted by a framework in each of three languages, and absent from most of the instrumentation libraries that wrap a provider SDK. **That is loop ownership, not language.**

The content switches are the best-named in the study

```
quarkus.langchain4j.tracing.include-tool-arguments = true
quarkus.langchain4j.tracing.include-tool-result    = true
quarkus.langchain4j.tracing.include-prompt         = true
quarkus.langchain4j.tracing.include-completion     = true
```

Four separate, self-describing properties, one per thing you might not want to log. Compare OpenInference's nine `hide_*` flags with inverted polarity, or an environment variable you have to know exists. This is what the report's "discoverable switch" argument looks like when someone gets it right.

Two problems, both worth knowing

It emits attribute names the registry removed. `include-prompt` and `include-completion` produce `gen_ai.prompt` and `gen_ai.completion`. Those were replaced by `gen_ai.input.messages` and `gen_ai.output.messages`, and the extension has no support for the replacements. Measured: both old keys present, neither new one. So the content is captured and a query written against the current conventions finds none of it — the same shape of failure as OpenInference, arrived at by a different route.

It cannot call the current model. The extension always sends `top_k`. Claude Opus 5 removed the sampling parameters and returns `400 "top_k is deprecated for this model"`. There is no way to suppress it, so this example runs Sonnet 4.6 while every other runs Opus 5. That is a property of the extension, disclosed rather than hidden.

And the MCP seam

This example gives its agent **local** tools. The demo repository behind this study also runs an otherwise identical agent whose tools arrive over MCP, and there the tool spans record a name and **no content** — with `include-tool-arguments` and `include-tool-result` both still true. Same library, same flags, same workload; the content is lost at the protocol boundary. Part 4 returns to that boundary and to who now owns it.

PART 3 · PROFILE

Agents you run rather than build

Everything so far is a library you import. Two of the agents measured here are products you install, where you cannot change the instrumentation — you get what the vendor ships. Both already speak OTLP, and one of them is more conformant than most of the libraries.

What was done

Neither tool's own configuration was touched. Claude Code was run with a `--settings` override pointing at the harness collector; Goose was run with environment variables only. Each was given the same job three times: a question with no tools, a question that forces a tool call, and the same tool call with content capture opted in.

CONDITION	GOOSE 1.48.0	CLAUDE CODE	WHAT SEPARATES THEM
No tools, default config	**11** `gen_ai.*`	5 `gen_ai.*`	Goose uses `gen_ai.usage.*`; Claude Code uses private `input_tokens`
Tool call, default config	**13**	6	Goose adds `gen_ai.tool.name` and `gen_ai.tool.call.id`
Tool call, content opted in	**17**	**6 — ignores the variable**	**Goose adds `gen_ai.tool.call.arguments` and `gen_ai.tool.call.result`**

The coding agent on a laptop beats five of the seven libraries

At 17 attributes with `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=true`, Goose records both the tool arguments and the tool result under the names the specification defines. Only one library in this study does the same. And it defaults content **off**, which is what the conventions ask for and which most libraries get wrong in the other direction.

This moved during the study. Goose 1.47.0 scored 8 / 10 / 14 on the same three conditions; 1.48.0 scores 11 / 13 / 17, having gained `gen_ai.agent.name`, `gen_ai.response.finish_reasons` and `gen_ai.response.id`. Every number in this report is a measurement of one version on one day, and this page is the proof.

Claude Code made a different bet

Judged on span attributes alone it looks thin. Judged on its **log stream** it is the richest thing measured anywhere in this study: typed events for `user_prompt`, `api_request`, `assistant_response`, `mcp_server_connection` and hook execution. Neither approach is wrong. One bets on attribute conformance so portable queries work; the other bets on event richness so a session can be replayed. A backend that only indexes span attributes will badly undercount the second.

Check this before pointing a coding assistant at a shared backend

Claude Code puts `user.email`, `user.id`, `user.account_uuid` and `organization.id` on **every span**. That is defensible for a tool whose whole value is per-developer attribution, and it is exactly the kind of thing you want to know before the telemetry lands in a shared tenant. A collector-side redaction processor is the answer if your policy needs one.

PART 3 · PROFILE

kagent: a platform, not a library

The last category is an agent platform running in Kubernetes. It behaves nothing like the seven libraries, and the difference is instructive rather than a defect.

What was done

A kind cluster, kagent installed by Helm with its OTLP endpoint pointed at the harness collector through `host.docker.internal`, and one agent invocation. One adaptation was needed: kagent's default `ModelConfig` targets OpenAI, so an Anthropic `ModelConfig` was added alongside rather than editing the Helm-managed one. The cluster was deleted afterwards.

One question produced 67 spans across 2 services and 7 instrumentation scopes

SCOPE	WHAT IT DESCRIBES
`kagent`	Its own Go controller
`gcp.vertex.agent`	The Python runtime, built on Google's Agent Development Kit
`a2a-python-sdk`	Agent2Agent protocol: event queues, task dispatch
`opentelemetry.instrumentation.anthropic`	**The official OpenTelemetry GenAI instrumentation**
`fastapi`, `httpx`, `otelhttp`	Transport and web-framework plumbing

That fourth scope matters more than the count. **kagent is the only thing in this study that ships the reference GenAI instrumentation inside a real product** rather than as a test case. The scope name is the proof, and it is the strongest evidence available that the reference implementation is usable in production.

Two things nothing else here does

- **It names the agent.** Thirteen convention attributes, including `gen_ai.agent.id`, `gen_ai.agent.name`, `gen_ai.agent.description` and `gen_ai.task.id`. A platform running many agents has to say which one ran, and it says so with specification names rather than private ones. Mastra is the only other subject in this study that names its agent this way, and it emits the first two of those four.
- **It records failure in span status.** The invocation measured actually failed, and kagent marked it: **6 spans Error, 26 Ok, 35 Unset**. Every one of the seven libraries marks a failing tool call as `Ok`. This is the only telemetry in the study where you can find a broken run by filtering on status.

The cost of the category

Sixty-seven spans for one question, most of them describing kagent's own plumbing. That is not a defect — it is what running agents as distributed infrastructure looks like, and the same trade-off any service mesh makes. But the GenAI signal has to be filtered out of framework noise, and the filter is the instrumentation scope. Part 2's provenance rule and this page rely on exactly the same field.

PART 3 · PROFILE

Three more agents you install

An SRE agent, a workflow platform and a Kubernetes diagnostic tool. None of them is a library you import, all three are things a team already runs, and between them they show the whole range: good structure with no vocabulary, and no telemetry at all.

HolmesGPT 0.40.0 — the conventions, one version behind

An SRE agent that investigates alerts. OpenTelemetry activates the moment `OTEL_EXPORTER_OTLP_ENDPOINT` is set (`holmes/core/tracing.py:398`), with no Kubernetes cluster required. One `holmes ask` produced **7 spans** under scope `holmesgpt` carrying **5** convention attributes.

WHAT IT DOES	MEASURED
Records token usage	`gen_ai.usage.input_tokens`, `.output_tokens`, `.total_tokens` — spec-named
Names the provider	`gen_ai.system`, which the conventions **removed** in favour of `gen_ai.provider.name`
Names the model-call span	`gen_ai.chat` — the attribute namespace used as a span name, where the spec asks for `chat {model}`
Identifies the tool	`holmesgpt.tool.name` and `.status`. No `gen_ai.tool.*`, no arguments, no result
Names the root span	The entire command line, **including the operator's question**

The root span name is the problem worth fixing first

A span name is meant to be low cardinality: it is what you group by. Putting the free-text question in it makes every investigation its own name, which defeats grouping and puts operator-typed text somewhere nobody expects to sanitise. The information is worth keeping — as an attribute, not as the name. `OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT=true` was set for this run and changed nothing.

Its metrics are more conformant than its spans

HolmesGPT emits five metrics, and two of them are exactly the ones the conventions define: `gen_ai.client.operation.duration` and `gen_ai.client.token.usage`. Before this, OpenLIT was the only thing in the study to emit either. The other three — `holmesgpt.tool.call.count`, `.duration` and `.errors` — are private but sensibly named. A team asking "what does this agent cost and how often do its tools fail" is well served here; a team asking "which tool did it run and what came back" is not served at all.

n8n — the best structure, the least meaning

A workflow platform with an AI Agent node and native OTLP tracing behind `N8N_OTEL_ENABLED`. One agent run produced **8 spans** under scope `n8n-workflow` and **zero** `gen_ai.*` attributes. What it does emit is a genuinely complete picture of the run:

SPAN	N8N.NODE.NAME	N8N.NODE.TYPE	WHAT A GENAI QUERY WOULD FIND
`workflow.execute`	—	—	nothing
`node.execute`	AI Agent	`agent`	nothing
`node.execute` ×3	Anthropic Chat Model	`lmChatAnthropic`	no model name, no tokens
`node.execute`	list_records	`toolCode`	no arguments, no result
`node.execute`	archive_records	`toolCode`	no arguments, no result

Read as a trace, that is exactly right: three model calls, two tool calls, correctly nested under the agent node. Read as GenAI telemetry, it is empty. The model is never named, no token is counted, and a tool call is indistinguishable from any other node. n8n has solved the hard half of the problem and not attempted the easy half.

Two things worth knowing before you try this

The CLI emits nothing. `dist/modules/otel/otel.module.js` declares `instanceTypes: ['main','worker','webhook']`, so `n8n execute` produces no telemetry at all — the workflow has to run inside the server. **And the licence gate did not fire.** The documentation states OTel tracing needs an Enterprise licence; this instance reported `isValid: false` and `entitlements: 0`, and traced anyway.

k8sgpt — a measured absence

k8sgpt emits no OpenTelemetry, and no cluster was needed to establish that. At commit `ebd79ac` (30 August 2026), **zero of its 199 non-vendor Go files import** `go.opentelemetry.io`, there is no tracer or span construction anywhere in `pkg/` or `cmd/`, and the string `OTEL_` does not appear in the repository. The k8sgpt-operator, at commit `97b3def`, is the same. The OpenTelemetry modules that appear in its `go.mod` are all marked `indirect`: they arrive through the Kubernetes and gRPC client libraries and nothing in k8sgpt calls them. An empty cell in this report is a finding, and this is one.

PART 4 · CONFORMANCE

Span names, kinds and status

Three rules from the written specification, checked against every span captured. These are the cheapest possible conformance tests and five of seven libraries fail at least one.

LIBRARY	SPAN NAME EMITTED	NAME	KIND	STATUS
Pydantic AI	`chat claude-opus-5 · execute_tool list_records · invoke_agent agent`	PASS	PASS	PASS
OpenTelemetry GenAI	`chat claude-opus-5`	PASS	PASS	PASS
OpenLIT	`chat claude-opus-5`	PASS	PASS	FAIL
CrewAI + OpenLIT	`invoke_agent check_max_iterations`	FORM ONLY	PASS	FAIL
OpenLLMetry	`anthropic.chat`	FAIL	PASS	FAIL
OpenInference	`messages.create`	FAIL	FAIL	FAIL
LangChain + OpenInference	`ChatAnthropic · model · tools`	FAIL	FAIL	FAIL

Why each failure matters

Five of seven set status to `Ok`, and the specification forbids it

The trace API is explicit: instrumentation libraries "SHOULD NOT set the status code to `Ok`" and should leave it `Unset` unless there is an error. `Ok` is reserved for an application developer asserting success. There is a neat demonstration inside the OpenLIT process: its own spans carry `Ok` while the `opentelemetry-instrumentation-httpx` spans it silently pulls in carry `Unset`. Contrib follows the rule; the library wrapping it does not, in the same trace.

OpenInference marks a remote API call INTERNAL

The specification allows `INTERNAL` only for models running in the same process. These calls go over HTTPS to a third-party API. Span kind drives client/server topology in most backends, so those calls disappear from dependency views — a functional consequence, not a cosmetic one.

Span names leak implementation details

`messages.create` is an SDK method. `ChatAnthropic` is a class name. `model` and `tools` are internal graph nodes. None of them identifies a class of operation a reader would recognise, which is what the cardinality guidance asks for. CrewAI is the subtle case: `invoke_agent check_max_iterations` has exactly the right *form* and the wrong *meaning*, which is harder to catch because it validates cleanly.

The same stop condition, three spellings

Every library writes a correctly named `gen_ai.response.finish_reasons`. The values disagree:

LIBRARIES	MODEL ASKED FOR A TOOL	MODEL FINISHED
OpenInference, OpenLIT, CrewAI	`tool_use`	`end_turn`
OpenLLMetry, Pydantic AI	`tool_call`	`stop`
OpenTelemetry GenAI	`tool_calls`	`stop`

Key-level convergence is real. Value-level convergence has not started. Seven services can all emit the right attribute name and still make a single cross-service query impossible — and the collector's normalizer only folds values for `gen_ai.operation.name`, not for finish reasons.

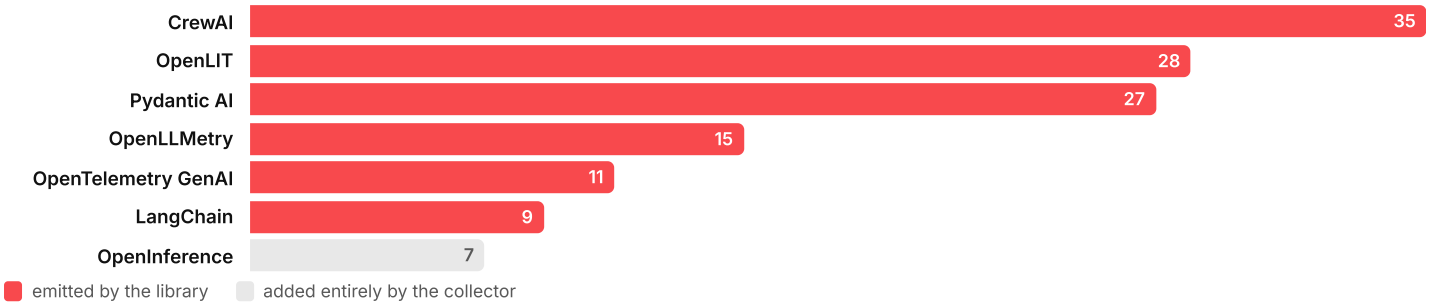
PART 4 · CONFORMANCE

Attribute coverage, and what translation fixes

Counting attributes is a weak measure on its own. The useful split is between what a library emits itself and what a collector had to add for it.

Convention attributes per library

Distinct `gen_ai.*` keys observed across all runs



OpenInference is the outlier in both directions

It produces the most raw data of anything tested — 36 to 42 attributes on a single model-call span — and **not one of them is a convention attribute**. All seven `gen_ai.*` keys on those spans were written by the collector's normalizer. Without that processor in the pipeline, an OpenInference service is invisible to any query written against the specification.

What normalization can and cannot fix

The `gen_ai_normalizer` processor rewrites **span attributes**. Nothing else. The measurement makes the limit concrete: it correctly derived `gen_ai.operation.name = chat` on the OpenInference spans, while those same spans remained named `messages.create`, kind `INTERNAL`, status `Ok`.

> **Normalization buys attribute parity, not span parity**
>
> A fully normalized OpenInference span is still non-conformant in name, kind and status, and no collector processor in the pipeline changes that. Any query that groups by span name or filters by span kind still sees seven different vocabularies. Translation solves the attribute problem and leaves the identity problem untouched.

Two things translation silently drops

- The **tool result**. OpenInference stores it in `output.value` and the mapping table has no entry for `gen_ai.tool.call.result`, so it never crosses.
- The **reasoning content**. OpenInference records the model's thinking in an indexed array form that the processor's own documentation states it does not reconstruct. The one library that captures the most reasoning detail stores it in the one shape translation refuses to read.

PART 4 · THE NORMALIZER

Translating vocabularies in the collector

One collector component exists specifically because these libraries disagree. Understanding what it fixes — and what it provably cannot — explains most of the gap between a library's raw output and what a portable query can see.

What it is

The `gen_ai_normalizer` processor rewrites AI attribute names from a library's private vocabulary into the OpenTelemetry GenAI conventions, in flight, without touching your applications. It was accepted into the collector's contrib distribution in June 2026 and is still marked **alpha**. Two source vocabularies are built in: OpenInference and OpenLLMetry.

The configuration is four lines, and that brevity is the entire argument for its existence:

```
processors:
  gen_ai_normalizer:
    sources:
      - name: openinference
      - name: openllmetry
```

How it works

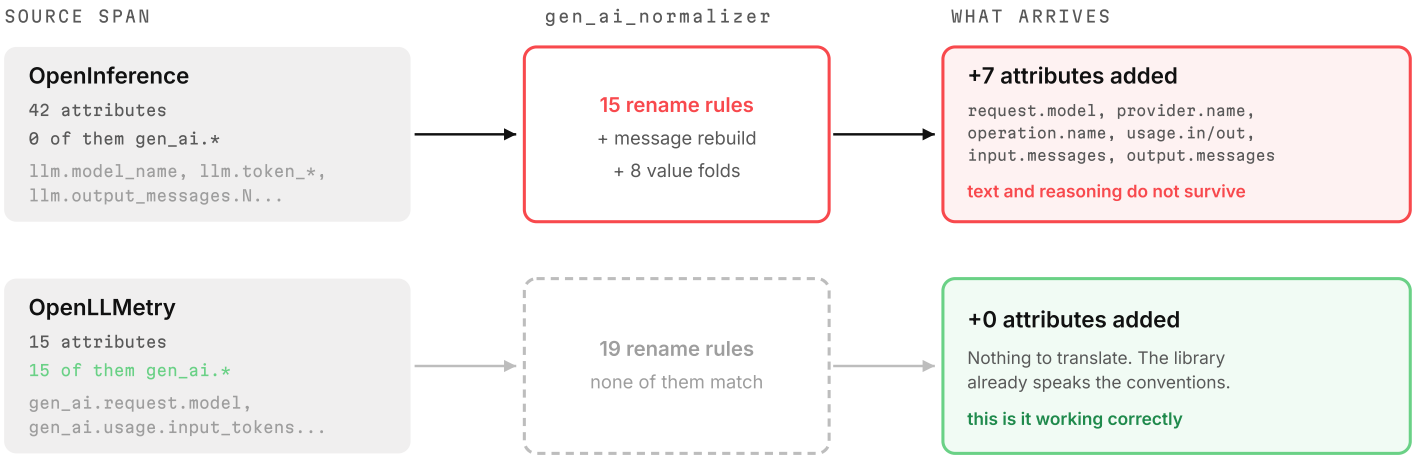
- **Sources are ordered, and every span passes through every source.** A span carrying two vocabularies gets both mappings applied.
- **Originals are kept by default.** `remove_originals` defaults to false, so a normalized span carries both the private name and the convention name. That is what makes the before-and-after in this report readable, and it is worth keeping in production too: the attributes it could not map are exactly the ones that survive.
- **Types are enforced after renaming.** A value that cannot be coerced safely to the target's spec type is dropped rather than mangled — a token count arriving as the string `"1,024"` produces no attribute at all, not a zero.
- **Some values are folded, not just keys.** Where a mapping lands on `gen_ai.operation.name`, the value is translated too: OpenInference's `LLM` becomes `chat`, `TOOL` becomes `execute_tool`.
- **It is traces-only.** Metrics and logs pass through untouched, so the specification's evaluation event — defined as a log record — is invisible to it no matter where you put it.

It rewrites attributes. Nothing else.

Span name, span kind and span status are never modified. This is visible in the measurement: the processor correctly derived `gen_ai.operation.name = chat` on OpenInference spans that remained named `messages.create`, kind `INTERNAL`, status `Ok`. **Normalization buys attribute parity, not span parity.**

What it does for each source

The two built-in sources need opposite amounts of work, and measuring both is the clearest demonstration of how unevenly this ecosystem has converged.



Measured on the runs behind this report. Both rows are the processor behaving exactly as designed.

For OpenInference: it does nearly everything

OpenInference uses none of the OpenTelemetry names, so the processor is doing all the work. Fifteen rename rules cover tokens, model, provider, tool name and description, tool call id and arguments, agent name and session id. Eight value folds translate `openinference.span.kind` into the operation enum. And the piece that earns its keep: **message reconstruction**, turning flattened indexed attributes such as `llm.input_messages.0.message.role` back into a single structured JSON message array. That is not a rename, and it is genuinely unpleasant to write by hand.

For OpenLLMetry: it does nothing, correctly

Nineteen rename rules are defined, and on the spans measured here **not one of them fired**. OpenLLMetry's Anthropic instrumentation already emits fifteen attributes, all of them convention attributes. A processor that adds nothing looks identical to a processor that is not running, which is why the report's method keeps originals in place: a span carrying both vocabularies is the proof it is working.

Three things it does not carry across

- **The tool result.** The OpenInference table maps the tool call's arguments and has no entry for the result, which stays untranslated in `output.value`.
- **The model's words.** OpenInference records text and reasoning in an indexed content array the processor documents that it does not reconstruct. 2,454 characters in, zero out.
- **Finish reasons, for OpenInference.** OpenLLMetry gets two mappings onto `gen_ai.response.finish_reasons`. OpenInference gets none, despite emitting `llm.finish_reason`.

Why not just use the transform processor

A fair objection, and one raised during the component's own donation review: the collector already ships a general-purpose attribute rewriter. Why maintain a dedicated one for this?

The size argument

The `transformprocessor` rewrites any attribute using OTTL, a small expression language. It can express these renames. The component author's comparison was concrete: the equivalent OTTL configuration is **98 statements across 152 lines of YAML**, because each of the 42 key mappings needs a `set` plus a `delete_key` with a `where` clause — against six lines for the dedicated processor.

transformprocessor

98 OTTL statements · 152 lines · you author and maintain every mapping · and message reconstruction is not expressible as a rename at all.

gen_ai_normalizer

6 lines · mappings ship with the component · maintained by the project that owns the target conventions · message reconstruction built in.

The argument that actually carried it

The reviewers' objection was governance rather than utility: if GenAI gets a dedicated normalizer, why not one for FAAS, for databases, for every convention area? The answer is that the fragmentation does not exist at this scale anywhere else. HTTP conventions are stable and universally implemented. GenAI has official instrumentation covering roughly seven libraries against thirty or more each for OpenInference and OpenLLMetry, using incompatible names for identical concepts. This report is a measurement of exactly that.

Where each one belongs

USE	REACH FOR
Renaming a known source vocabulary onto the conventions	`gen_ai_normalizer`, with a built-in source
A vocabulary it has never heard of	`gen_ai_normalizer`, with a user-defined source and your own `mappings`
Changing a value without renaming the key	`transformprocessor` — the normalizer only folds values on `gen_ai.operation.name`
Forcing a uniform shape on the free-form message attributes	`transformprocessor`, paired after the normalizer
Reconciling finish-reason *values* across libraries	`transformprocessor` — nothing else does this today
Translating between versions of the OTel conventions	Neither. `schemaprocessor` would, but the source conventions publish no schema files.

The ordering that catches people

Put `gen_ai_normalizer` **before** anything that reads `gen_ai.*`, and before `batch`. An OpenInference service emits zero convention attributes on its own, so any processor placed ahead of the normalizer sees nothing to match on and silently does nothing.

The same translation, moved into the client

Part 4 has argued that translating between AI vocabularies belongs in the collector. One project disagrees in the most concrete way available: it ships the same translation table inside its SDK, pointed at its own data model instead of at the specification.

What Langfuse emits

Langfuse is an observability vendor whose SDK is built on OpenTelemetry, which makes it easy to assume it speaks the conventions. Measured, it does not. One agent run through `langfuse` 4.15.1 produced **4 spans** under scope `langfuse-sdk`, carrying six attribute names and **zero** `gen_ai.*`:

`langfuse.observation.type` · `langfuse.observation.input` · `langfuse.observation.output` · `langfuse.observation.model.name` · `langfuse.observation.usage_details` · `langfuse.internal.is_app_root`

Span names are whatever you pass, all spans are `INTERNAL`, and status is never set. Judged against the specification that is a poor showing. But judging it that way misreads the product: Langfuse is a backend that ships an SDK, and the SDK's job is to describe your code in *Langfuse's* model — trace, observation, generation, tool. Conformance was never the goal.

The interesting half: what it reads

Because Langfuse expects to receive telemetry from instrumentation it did not write, its exporter carries a hardcoded table of the attribute names other people's libraries use for input and output. Read out of `_client/span_exporter.py` in the installed package, it recognises **45 attribute names across ten vocabularies** and folds them all into two Langfuse fields:

VOCABULARY	KEYS	EXAMPLES
Vercel AI SDK	**10**	`ai.prompt.messages`, `ai.toolCall.args`, `ai.response.toolCalls`, `ai.result.text`
Unprefixed guesses	7	`input`, `output`, `prompt`, `tool_arguments`, `tool_response`, `final_result`
LiveKit Agents	6	`lk.chat_ctx`, `lk.user_transcript`, `lk.function_tool.output`
OTel GenAI conventions	**5**	`gen_ai.input.messages`, `gen_ai.tool.call.arguments`, `gen_ai.tool.call.result`
Google ADK	4	`gcp.vertex.agent.llm_request`, `gcp.vertex.agent.tool_response`
Langfuse itself	4	`langfuse.observation.input`, `langfuse.trace.output`
OpenInference	2	`input.value`, `output.value` (plus `llm.input_messages` by prefix)
Firebase Genkit	2	`genkit:input`, `genkit:output`
MLflow	2	`mlflow.spanInputs`, `mlflow.spanOutputs`
Traceloop / OpenLLMetry	2	`traceloop.entity.input`, `traceloop.entity.output`
Pydantic AI	1	`pydantic_ai.all_messages`

Two answers to one problem

This is the strongest available evidence that the fragmentation this report measures is real. Langfuse did not build that table for fun; it built it because the spans arriving at its ingest genuinely use all of those names. The `gen_ai_normalizer` processor exists for the same reason. The two differ in where the work happens and what it targets.

	GEN_AI_NORMALIZER	LANGFUSE SDK EXPORTER
Where	The collector, once for the estate	Every application, in-process
Target	The OpenTelemetry conventions	Langfuse's own two fields
Extensible	Yes — user-defined sources in YAML, as Part 4 does for Spring AI	No — the table is compiled in
Portable result	**Any OTLP backend can query it**	**Only Langfuse can**
Preserves detail	Attribute by attribute	Collapses everything to input and output

Neither is wrong. If Langfuse is your backend, a compiled-in table you never configure is a real advantage. If you have more than one backend, or expect to change one, the collector is the only place the translation survives that change. The two are also composable: normalize toward the specification in the collector, and Langfuse's `gen_ai.*` row picks it up on arrival.

PART 4 · GOVERNANCE

Two donations, one accepted

Both of the vocabularies the collector translates were offered to OpenTelemetry. Only one arrived. The record is public, and it resolved in a direction that is hard to predict from the code alone.



Both issues were closed on the same day, 16 June 2026 — one by acceptance, the other by abandonment. Sources: `open-telemetry/community` issues #2571 and #3467.

OpenLLMetry: proposed February 2025, closed unlanded

Traceloop opened community#2571 on 13 February 2025, offering more than 40 instrumentations spanning foundation-model providers, AI frameworks and vector databases — Apache 2.0, in production since October 2023, with “a team of GenAI domain experts” offered for co-maintenance.

Three months later OpenTelemetry's technical committee — the group that reviews proposals of this kind — asked three practical questions: what trademarks were included, whether copyright could transfer to “The OpenTelemetry Authors”, and whether the donation would proceed instrumentation by instrumentation under the LLM SIG's oversight. **Those questions were never answered.** Someone asked for an update in July 2025. Someone asked again in June 2026. The whole issue drew four comments in sixteen months, and the last one closed it:

> “Unfortunately this never landed. Closing now that the OpenInference donation is completed. For anyone working on OpenLLMetry, we would love to work with you in the GenAI SIG!”
>
> TRASK · 16 JUNE 2026

OpenInference: proposed May 2026, accepted 24 days later

Arize opened community#3467 on 23 May 2026 offering instrumentation code across Python, JavaScript, Java and Go, with the Python packages then running between 75,000 and 5.7 million monthly downloads. Five days later the TC was working through CNCF paperwork; ten days after that the mechanism was agreed. The Governance Committee, which holds the final vote on donations, accepted on 16 June after a due-diligence review with no objections. The GenAI SIG — the special interest group that maintains these conventions and instrumentations — attached conditions, covered on the next page.

A code grant is a snapshot, not a merger

A maintainer asked the useful clarifying question — what is the difference between a grant and a transfer, and would OpenTelemetry be mirroring Arize's changes indefinitely? The answer sets expectations precisely:

“The donation is just a starting point for the GenAI SIG to bootstrap our own GenAI instrumentations in areas where we currently have gaps. We are not currently planning to mirror changes in either direction.” The donated repository was archived and the SIG cherry-picks from it. OpenInference continues as its own product. **The grant covers the instrumentation code only, explicitly not the OpenInference specification or its semantic conventions.**

PART 4 · GOVERNANCE

What the acceptance actually promised

The conditions attached to the OpenInference donation are the most useful public document in this whole area, because they are an admission of what the conventions do not yet cover — written by the people who own them.

The conditions, verbatim

> GenAI SIG maintainers must show commitment to incorporate instrumentations and agree to work towards covering convention gaps such as:
>
> - Supporting additional operation and event types — **guardrails, reranking, evaluations, handoffs**
> - Defining attributes for additional input and output parameters
> - Specialising conventions for additional inference API flavours: realtime, audio, and other stream-based binary content generation
>
> LMOLKOVA · COMMUNITY#3467 · 16 JUNE 2026

Read that as a roadmap and it is more than most standards efforts publish. Read it as a gap analysis and it says plainly that the conventions do not yet describe an agent — only the model call underneath one. **And note what is not on the list:** the tool call's result, the single attribute this report finds missing from six of seven libraries.

The measured irony

Set the governance record beside the telemetry measured for this report and they point in opposite directions:

PROJECT	CONVENTION ATTRIBUTES IT EMITS ITSELF	NEEDS THE COLLECTOR TO TRANSLATE IT	DONATION OUTCOME
OpenLLMetry	**15 of 15 — fully converged**	**no**	NEVER LANDED
OpenInference	0 of 42 — entirely private	yes	ACCEPTED

The project whose instrumentation already speaks the OpenTelemetry conventions is the one whose donation never landed. The project whose vocabulary is entirely its own, and which needs a dedicated collector processor to be queryable as OpenTelemetry at all, is the one now inside OpenTelemetry.

Nobody behaved badly here

Donations need sustained attention from people with authority to sign things, and sixteen months is a long time to hold that. Arize answered every question within days; Traceloop answered none. But the outcome corrects a tidy assumption worth naming: **convergence in code and convergence in governance are separate processes and can run in opposite directions.** "Donated to OpenTelemetry" and "speaks OpenTelemetry" are not the same signal, and this report measured them disagreeing.

Why this matters when you choose a library

Governance tells you where maintenance effort will land over the next few years; measurement tells you what your telemetry looks like today. On the evidence here you need both, and they will not always agree. OpenInference's instrumentations are being cherry-picked into `opentelemetry-python-genai` — the reference implementation profiled earlier in this report, and currently the most conformant and least informative option tested. Watching that package gain OpenInference's coverage while keeping the specification's defaults is the single most useful thing to track from here.

PART 4 · GOVERNANCE

The protocols moved to a different foundation

While the GenAI conventions were being written inside OpenTelemetry, the protocols agents actually speak consolidated somewhere else. Understanding that split explains a set of gaps this study kept running into — and who is now working on them.

The Agentic AI Foundation

AAIF launched under the Linux Foundation on 9 December 2025, co-founded by Anthropic, OpenAI and Block, as a neutral home for open agent standards. Platinum members include AWS, Anthropic, Block, Bloomberg, Cloudflare, Google, Microsoft and OpenAI. It now hosts the protocol layer of this ecosystem:

PROJECT	WHAT IT IS	APPEARS IN THIS STUDY AS
Model Context Protocol	**How an agent is handed tools it did not ship with**	**The boundary where tool content is lost**
Agent2Agent (A2A)	**An open protocol for agents to communicate and exchange tasks across platforms**	**Seven of kagent's spans, via `a2a-python-sdk`**
goose	An extensible coding agent	The most conformant coding agent measured
agentgateway	Connections between agents, models, tools and APIs	—
AGENTS.md	An open format for guiding coding agents	—

Two foundations, one seam

The **protocols** agents use to reach tools and each other live in AAIF. The **vocabulary** for describing what they did lives in OpenTelemetry. Both sit under the Linux Foundation and neither owns the other. That is a reasonable division — a transport protocol should not dictate a telemetry schema — but the seam between them is where this study kept finding problems:

- **Content stops at the MCP boundary.** The same LangChain4j agent, same flags, records tool arguments and results for local tools and only a tool name over MCP.
- **Trace context is not free over MCP.** It propagates only because `traceparent` rides in the request's metadata and the server reads it. Where an implementation does not, the client and server halves of one tool call land in separate traces.
- **A2A brings its own span vocabulary.** kagent produced `a2a.server.events.event_queue.EventQueue.dequeue_event` and similar — accurate about the protocol, carrying no GenAI meaning.

That seam is not unowned. It has a working group.

AAIF's **Observability & Traceability Working Group** was ratified 15 April 2026 and approved 13 May 2026. Chaired by Pavan Sudheendra (Cisco) with Matthew Lee (Datadog) as co-chair, it meets biweekly and works in the open at `github.com/aaif/wg-observability-and-traceability`. It is one of seven AAIF working groups, alongside a cross-group taxonomy workstream.

PART 4 · GOVERNANCE

The working group, and what it is aiming at

The charter is worth reading in full if you work in this area. It states this report's central thesis independently, and its founding focus area names the exact gap the measurements found.

It is explicitly not a new standards body

"The observability landscape already has active standards efforts (OpenTelemetry GenAI SIG, OWASP AOS, W3C Trace Context, and others). Creating a competing specification body would worsen the fragmentation problem the WG exists to solve." Its default action is to coordinate and contribute upstream; new specifications only where a gap analysis shows nobody is addressing a need. Given the XKCD-standards risk in this space, that restraint is the most reassuring line in the document.

The thesis, in their words

The charter names three reasons the group exists. The second is this report in one sentence:

> "Existing standards (OTel GenAI semantic conventions, OpenInference, OpenLLMetry) provide good coverage of individual LLM calls. However, understanding agent behavior at the session or workflow level — across multiple turns, tool invocations, and model interactions — remains an open problem."
>
> OT-WG CHARTER · RATIFIED 15 APRIL 2026

Their mandate is stated as **agent-level observability**, explicitly distinct from and building on LLM-call-level telemetry, which they name as the OTel GenAI SIG's domain. That is exactly the division this report measured: the model call converged, the agent loop did not.

The founding focus area names the missing field

The **Agent Behavior Trace Model** is the group's first-priority work, covering three dimensions. Each maps onto a finding in this report:

TRACE MODEL DIMENSION	WHAT THIS STUDY MEASURED AGAINST IT
Execution structure sessions, turns, tool calls, hierarchy	Four of seven setups produce no run at all — one disconnected trace per model call
Reasoning and planning plans, options, strategies, reflection	Three libraries record that reasoning happened; none carry content, because the provider omits it by default
Outcomes and effects what the agent produced, what changed in the world, whether intent was fulfilled	**This is `gen_ai.tool.call.result`. Three of ten implementations emit it**

The charter's second focus group puts it even more directly: *"Intent, outcome, and side-effect attribution — what the agent requested, what the execution surface returned, and what externally visible state changes resulted."* That is the forensic question this entire report has been asking, written down as a work item.

Five focus groups, and where the measurements land

FOCUS GROUP	COVERS	RELEVANT FINDING HERE
Orchestration & coordination	Delegation and hand-offs, context propagation across agent/model/tool/protocol boundaries, session continuity, human-in-the-loop checkpoints, guardrail events	Trace shattering; MCP context propagation
LLM primitive observability	**Tool and skill execution, intent/outcome/side-effect attribution, retry and fallback when execution fails**	**The tool result; a failing tool marked `Ok`**
Protocol observability	**MCP, A2A and other protocols; protocol normalization; transport boundaries; alignment with OTLP export paths**	**Content lost at the MCP boundary**
State & context	Memory access patterns, context assembly, evidence references, session persistence	Not measured here
Agentic reasoning	Planning, evaluation and selection, reflection, reasoning boundaries — deliberately without mandating capture of private chain-of-thought	Reasoning parts present, content empty

Where to send this

Open issues include **#12 Plan Agent Behavior Trace Model recommendation** and **#29 Cross-boundary observability gap model for agentic systems**. Both are the right home for measurements of the kind in this report. The group works on GitHub, meets biweekly on Wednesdays, and states that "GitHub issues are the source of truth for durable status". Anyone running this stack in production has something the group explicitly says it needs: evidence of where the gaps actually bite.

PART 5 · WHAT IT SAYS

Reading the model's own words

The most useful thing in the whole specification is not about tools. It is the model's answer, captured as text. Whether you can find it, and under what name, splits these libraries in a way nothing else does.

Why the answer text matters more than it sounds

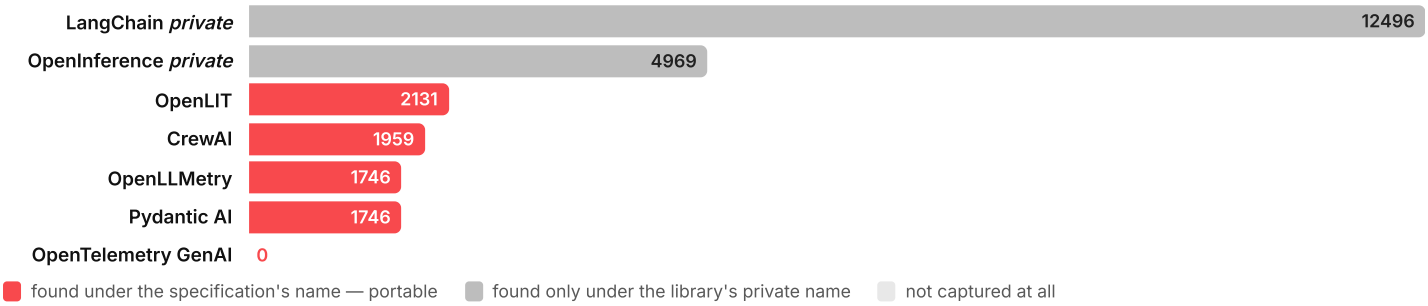
Token counts tell you what a run cost. Latency tells you how long it took. Neither tells you whether the agent did a good job. For that, something has to read what the model actually said — and the standard way to do that is a second model, an **LLM-as-a-judge**: you hand it the question, the answer, and a criterion such as "is this relevant" or "does this leak personal data", and it scores the response.

That technique needs one thing: **the response text**. So the practical question for any instrumentation library is simply whether the model's words end up in your telemetry, and whether a portable query can find them.

> **Two different attributes, easily confused**
>
> `gen_ai.output.messages` is the **specification's** name for the model's response, structured as JSON. `output.value` is **OpenInference's own** name for roughly the same thing. Both may contain the full answer. Only the first is portable — a query written against it works no matter which library produced the span. That difference is the whole of this page.

Where the model's words actually end up

Characters of readable text captured, by attribute name. Same agent, same answer, seven libraries.



The result that surprised us

The two libraries that capture **the most** content are the two where a standard query finds **none** of it. LangChain with OpenInference records 12,496 characters of the model's response — roughly six times the best spec-named library — and stores every character in `output.value`. Under `gen_ai.output.messages`, the same span offers zero characters of text.

This is not the library failing to record things. It is a translation gap: OpenInference stores the answer in an indexed, flattened form, and the collector's normalizer documents that it does not reconstruct that particular shape. So the content is captured, survives to the collector, and is dropped on the way to a portable name.

> **What this means in practice**
>
> If you run OpenInference or LangChain and write your evaluation against `gen_ai.output.messages`, you will get empty results and conclude nothing was captured. The data is there, one attribute away, under a name only that library uses. **Query `output.value` alongside the standard attribute until this closes.**

Reasoning is a special case, and the answer is surprising

Three libraries record that the model reasoned — OpenLLMetry as a `reasoning` part, Pydantic AI as a `thinking` part, OpenInference as a reasoning content block with a cryptographic signature. **In all three the reasoning content is empty**, because the provider returns thinking blocks with their text omitted by default. That is a request-side setting, not an instrumentation gap. Appendix B covers it in full.

And what the answer text still cannot tell you

Judging the *answer* needs the response text. Judging the *actions* needs the tool fields, and no quantity of prose substitutes for them: whether the agent picked the right tool, passed the right arguments, and what came back are facts about execution rather than about writing. That is the next page.

PART 5 · WHAT IT SAYS

Reconstructing the run

The real test is whether you could reconstruct what the agent did with the source code closed — which is the position you are in during an incident.

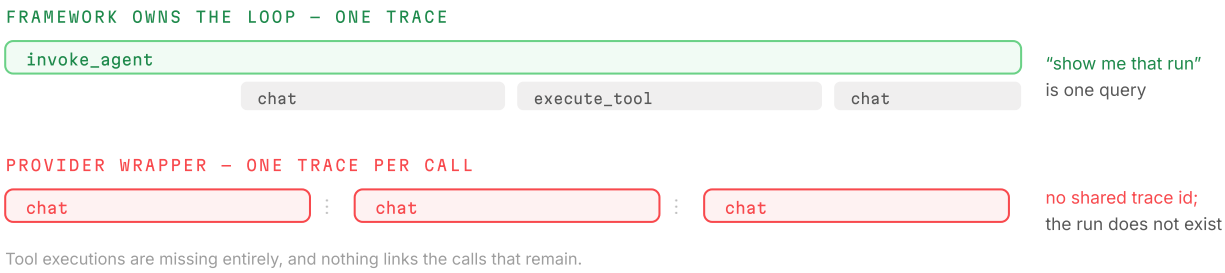
QUESTION	PYD AI	CREWAI	OPENLIT	LANGCHAIN	OPENLLM	OPENINF	OTEL
Which tool ran?	SPEC	SPEC	SPEC	SPEC	NO	NO	NO
With what arguments?	SPEC	SPEC	SPEC	PRIVATE	NO	PRIVATE	NO
What did it return?	SPEC	NO	NO	PRIVATE	NO	PRIVATE	NO
Why did it stop?	SPEC	SPEC	SPEC	NO	SPEC	NO	SPEC
What did it say?	SPEC	SPEC	SPEC	NO	SPEC	NO	NO
How much did it cost?	TOKENS	DOLLARS	DOLLARS	TOKENS	TOKENS	TOKENS	TOKENS

A failing tool is recorded as a success, everywhere

In the runs where the agent tried the archiving tool, it failed. Ten spans carry that failure. Every one of them is marked status `Ok`, with no `error.type` and no span events — including LangChain's `archive_records` span, the span for the failing call itself.

The fair caveat: the tool returned an error *payload*, it did not raise, so no library can be blamed for missing it. **That is precisely the argument for the missing field.** `gen_ai.tool.call.result` is the only thing that would let anything downstream notice, and six of seven do not emit it.

Provider hooks shatter the run into unconnected traces



Tool executions are missing entirely, and nothing links the calls that remain.

The four libraries that patch the provider SDK produce **one root trace per model call**, not one per run. Nothing joins them, because nothing owns the loop that spans them. You cannot ask a backend to show you "that agent execution" — no such object exists, only adjacent traces. For an incident that is arguably worse than a missing attribute, and it also means agent-level evaluation cannot be built by buffering a trace, because there is no shared trace to buffer.

Feeding a real evaluation framework

Reconstructing a trajectory from spans alone and handing it to DeepEval's tool-correctness metric produces three distinct failure modes, and all three are worse than an honest blank:

- **Not evaluable** — three libraries never say which tool ran, so no trajectory exists to score.
- **Scored zero for a correct run** — CrewAI and OpenLIT called the right tool with the right arguments; the strict comparison fails only because the result is absent. The agent is marked wrong for the instrumentation's omission.
- **A silent pass** — comparing an expectation with no arguments against telemetry with no arguments returns a perfect score. A run that recorded nothing scores 1.00.

So an evaluation pipeline fed standard GenAI telemetry does not produce missing answers. It produces confident wrong ones — which is why every evaluation framework ships its own decorator rather than reading the standard.

PART 5 · EVALUATION

What is actually worth evaluating

“Evaluate your agent” is six different jobs with six different data requirements. Separating them is the difference between a plan and a wish, because each one is blocked by a specific attribute this report has already measured.

WHAT YOU WANT TO KNOW	WHAT IT NEEDS	AVAILABLE?
Was the answer good? relevance, tone, format, harmfulness, PII	`gen_ai.output.messages`	4 of 7
Which tools, in what order? tool selection, step efficiency, plan adherence	`gen_ai.tool.name + one trace`	3 of 7
Were the arguments right? argument correctness	`gen_ai.tool.call.arguments`	3 of 7
Did the tool do the right thing? side-effect and outcome correctness	`gen_ai.tool.call.result`	1 of 7
Did it finish the job? task completion against a goal	`result + a stated goal`	1 of 7
What did it cost to succeed? tokens and dollars per completed task	`gen_ai.usage.* + outcome`	tokens: 7 of 7

Availability counted across the seven library setups at their defaults. Coding agents and the Java frameworks shift several rows upward.

The pattern in that picture

Evaluations of **what the agent said** are broadly possible today. Evaluations of **what the agent did** are mostly not. That is the same split the rest of this report found, restated as a purchasing decision: if the question you care about is quality of prose, the ecosystem is ready; if it is correctness of actions, you are choosing from a much shorter list of frameworks.

The rows in red are the ones that matter for a destructive agent

An agent with write access to a database is not primarily a prose-quality problem. “Did the tool do the right thing” and “did it finish the job” are the questions that decide whether it is safe to run unattended, and they are the two with the least telemetry behind them.

PART 5 · EVALUATION

agent-health, run against a real judge

OpenSearch's agent-health is the only tool in this study built specifically for agent *trajectories* rather than model outputs. It was installed, configured with an Anthropic judge, and run.

Getting it running

Its default judge is AWS Bedrock. The documented alternative is an `openai-compatible` provider, which needs an OpenAI Chat Completions endpoint — and Anthropic's native Messages API is not one. **LiteLLM is the adapter**: a container in front of Anthropic presenting the OpenAI shape, verified with a round trip before wiring anything else.

```
OPENAI_COMPATIBLE_ENDPOINT = http://localhost:4000/v1/chat/completions  // LiteLLM
litellm: anthropic/claude-sonnet-4-6, drop_params: true
```

One practical snag: the config must be `.mjs`, not the documented `.ts` — the CLI runs under plain node with no TypeScript loader, and a `.ts` config fails with "Unknown file extension". A bare config also replaces the built-in agents unless `extends: true` is set.

What a run produces

Four scores and a trajectory

`accuracy 95`, `faithfulness 99`, `latency_score 93`, `trajectory_alignment_score 90`, with written judge reasoning and a seven-step trajectory: `assistant → action → tool_result → assistant → action → tool_result → response`.

And one span, containing none of it

With telemetry enabled it emitted a single `test_case` span: `agent_health.agent.duration_ms`, `.connector.protocol`, `.judge.attempts`, `.judge.duration_ms`, `.judge.model`. **No `gen_ai.evaluation.*` at all.** The scores went to a local JSON file.

That distinction is worth stating plainly: agent-health's OpenTelemetry emission describes **the act of evaluating** — how long the judge took, how many attempts it needed — not **the evaluation result**. Useful for operating the evaluator, useless for querying quality alongside your other telemetry.

What it requires from your instrumentation

Its documented span requirements are specific, and this study's seven setups were scored against them:

SETUP	AGENT SPAN (4)	LLM SPAN (5)	TOOL SPAN (2)	MISSING
pydantic-ai	**4/4**	**5/5**	**2/2**	**nothing**
crewai	2/4	4/5	2/2	`system`, `conversation.id`
openlit	1/4	4/5	2/2	`system`, `conversation.id`, `agent.name`
langchain	1/4	4/5	2/2	`system`, `conversation.id`, `agent.name`
openllmetry	1/4	4/5	1/2	+ `tool.name`
openinference	1/4	4/5	1/2	+ `tool.name`
otel-genai	1/4	4/5	1/2	+ `tool.name`

It requires an attribute the registry removed

Every failure above shares one cause: agent-health requires `gen_ai.system`, which the conventions **replaced with** `gen_ai.provider.name`. Six of seven setups emit the new name. Its docs also ask for `gen_ai.tool.call_id` where the specification says `gen_ai.tool.call.id`. An evaluation tool built on the conventions, pinned to a superseded revision of them — which is the schema-drift problem this report keeps describing, arriving from the consumer side for once.

One thing it does not need

Its required tool attributes are the operation name and the tool name. **Not the arguments, and not the result.** So Golden Path comparison scores which tools ran and in what order, rather than whether they were called correctly or what came back. That is a lower bar than DeepEval's strict mode — and it is why agent-health can evaluate setups that DeepEval could not.

Two projects, two philosophies

Both OpenInference and OpenLIT support evaluation. They model it in fundamentally different places, and only one of them produces telemetry a third party can read.

OpenInference: evaluation as its own vocabulary

OpenInference defines a full evaluation attribute set — `evaluation.name`, `evaluation.score`, `evaluation.label`, `evaluation.explanation`, `evaluation.identifier`, `evaluation.metadata` — plus `EVALUATOR` and `GUARDRAIL` span kinds. It is a richer model than the specification's in one specific way: `evaluation.annotator_kind` records **whether a human or a model produced the score**, a distinction the OpenTelemetry conventions have no attribute for.

But no OpenInference instrumentation writes any of it. The constants are defined in its semantic-conventions package and used by Phoenix, Arize's platform, which attaches evaluations out of band after traces have been collected. From the instrumentation alone, evaluation telemetry never appears.

OpenLIT: evaluation as specification-conformant telemetry

OpenLIT has two separate paths, and they behave completely differently:

API	WHAT IT DOES	TELEMETRY PRODUCED
`openlit.eval()`	Posts prompt, response and context to the OpenLIT platform for server-side scoring	**None.** A plain HTTP POST. No tracer, no span, no attribute. The score exists only in the platform.
`openlit.score.record_evaluation_score()`	**Records a score against a span you identify**	**Emits `gen_ai.evaluation.result` as a log record carrying the target's `trace_id` and `span_id`, plus a span event when the span is still recording**

This is the specification's shape, implemented correctly

The conventions define `gen_ai.evaluation.result` as an Event in the logs data model, not as a span attribute, precisely so several evaluators can score the same operation without overwriting each other. OpenLIT emits exactly that, with `gen_ai.evaluation.name`, `score.value`, `score.label` and `explanation`. **It is the only project measured here that both defines and writes the specification's evaluation attributes.**

The detail that matters architecturally

OpenLIT resolves its target span from a raw `trace_id` and `span_id` using a non-recording span handle. In plain terms: **you can attach an evaluation result to a span that was exported minutes ago.** The correlation is carried by ids on the log record rather than by mutating the original span.

That solves the problem every "evaluate telemetry automatically" design runs into. The conventions recommend associating the evaluation with the operation it scores, which sounds like it requires holding the span open until the evaluator returns. It does not. A correlated log record emitted later achieves the same association, and removes the need for any synchronous enrichment step at all.

Three places to put the judge

Evaluation is one mechanism — a second model call with a rubric — and three quite different systems depending on where you run it. The further it sits from the decision, the less it can see and the more it depends on your telemetry being right.

CLOSER TO THE DECISION — SEES MORE, REACHES LESS		— SEES LESS, REACHES MORE
01 IN THE AGENT	02 IN THE COLLECTOR	03 IN THE BACKEND
A second model call, inside the run	**A processor reading telemetry in flight**	**A job over stored traces**
Sees everything in memory: prompt, tools, arguments, results — whether or not any of it reached a span.	Sees only what was emitted. Everything in Part 3 decides what is judgeable at all.	Sees history. Can re-score last month against a new rubric, compare releases.
The only one that can GATE.	One policy for every agent you run. No code changes.	**Cheapest to operate. Can never block anything.**
WHAT IT COSTS		
A second model call on every gated request, in user-visible latency. Judges one agent.	Content leaves the collector for an evaluator. Inherits every privacy constraint the spans carry.	Nothing on the request path. Scores land wherever the backend puts them.

Measured constraint: columns 2 and 3 can only evaluate an agent run where one exists. For four of the seven setups measured, provider-hook instrumentation emits one disconnected trace per model call, so there is no run to score.

The gradient is the whole decision

Closer to the decision means more visibility and narrower reach. Further away means broader reach and total dependence on the conventions being right. There is no position that is simply better, and most teams eventually run two: a narrow gate inline for the actions that matter, and a broad sampled job further out for trends.

Which one each tool in this study is

In the agent: pydantic-evals, DeepEval's decorators, and any guardrail you write yourself. **In the collector:** nothing ships today — this is the gap a `genaievaluationprocessor` would fill. **In the backend:** OpenLIT's auto-evaluation, Phoenix annotations, and agent-health's Golden Path suite.

The second axis: when it runs

Where the judge sits is only half the design. When it runs relative to the request is the other half, and the two get conflated constantly.

	OFFLINE before deploy, on a dataset	ONLINE, SAMPLED a percentage of production	INLINE, GATING every request, before it returns
IN AGENT	**Regression suite** pydantic-evals, DeepEval	**Possible, unusual** sample inside your own loop	**The guardrail** block, retry, escalate
COLLECTOR	n/a — no traffic yet	**The fleet policy** one config, every service	Impossible — the answer already returned
BACKEND	**Golden path suite** agent-health	**Quality trends** OpenLIT auto-eval, Phoenix	Impossible

Only the top-right cell can change what the user sees. Everything else informs a decision someone makes later.

Reading the grid

- **Offline is where a gold set belongs.** A curated dataset with human labels, run before deploy. It costs nothing in production and is the only place you can honestly calibrate a judge, because you have ground truth to check it against.
- **Online sampled is where cost lives.** Every judged request is a second model call you pay for, so the sampling rate is a budget decision rather than a technical one. Deterministic selection matters: hash the trace id, so retries and multiple collector replicas make the same choice.
- **Inline gating is the only one that changes an outcome** — and it puts a model call on your critical path and makes the evaluator a dependency of your agent's availability. Reserve it for actions you would not want to take back.

The trap in the middle row

A collector-side judge is attractive because it is one policy for every agent in the estate with no code changes. But it can only read what was emitted, and this report measured that: keyed on `gen_ai.output.messages` it would find nothing on three of seven setups, and keyed on `gen_ai.tool.call.result` nothing on six of seven. If you build it, emit a metric for "selected but no content" — otherwise a silent no-op looks exactly like a clean bill of health.

Judges are biased, and the mitigation is boring

Position bias, verbosity bias and self-enhancement bias are all documented for LLM judges. A score without a rubric, an evaluator version, a sample definition and human calibration is weak evidence whichever cell it comes from. The mitigation is a human-labelled gold set and an agreement statistic, not raw agreement percentage.

PART 6 · COMPARISON

Everything measured, side by side

One table. Every column is a measurement from the run, not a claim from documentation.

LIBRARY	SETUP	SPANS /RUN	GEN_AI ATTRS	SPAN NAME	KIND	STATUS	TOOL NAME	ARGS	RESULT	MODEL TEXT	REASON-ING	METRICS	SPEC EVENT	ONE TRACE
Pydantic AI	**2 lines**	**5**	**27**	✓	✓	✓	✓	✓	✓	**1746**	✓	—	—	✓
OpenLIT	1 line	2	28	✓	✓	✗	✓	✓	✗	2131	—	5	✓	✗
CrewAI + OpenLIT	~10 lines	26	35	~	✓	✗	✓	✓	✗	1959	—	5	✓	✓
OpenLLMetry	1 line	1	15	✗	✓	✗	—	—	—	1746	✓	3	—	✗
LangChain + OpenInf.	4 lines	8	9	✗	✗	✗	✓	priv	priv	0	—	—	—	✓
OpenInference	4 lines	1	7*	✗	✗	✗	—	priv	priv	0	priv	—	—	✗
OpenTelemetry GenAI	3 lines	1	11	✓	✓	✓	—	—	—	0	—	—	—	✗

✓ present and specification-named · priv = present under a private name · ✗ present and wrong · — absent · ~ correct form, wrong meaning. * OpenInference emits zero convention attributes itself; all seven were added by the collector's normalizer. "One trace" means the whole agent run shares a trace id.

Ease of use, as measured

- **One line to instrument:** OpenLIT and OpenLLMetry.
- **No package to install:** Pydantic AI, which emits OpenTelemetry natively.
- **Most switches to learn:** OpenInference, with nine separate `hide_*` flags and no single content control.
- **Install failures encountered:** OpenLLMetry ships a package that cannot be imported because it omits a declared dependency; OpenLIT pins an Anthropic SDK two major versions behind current.

What each cannot do

- **Pydantic AI:** no metrics, and it ties you to its framework.
- **OpenLIT / CrewAI:** no tool result, so no trajectory evaluation.
- **OpenLLMetry:** no tool information at all.
- **LangChain + OpenInference:** no model text in the normalized content field.
- **OpenTelemetry GenAI:** nothing about content or tools at its default.

Beyond the seven: everything else measured

The seven Python setups are a controlled comparison: same language, same provider SDK, same prompt, same three tools. Eight more subjects were measured that do not fit that frame — a TypeScript framework, a vendor SDK, two products you install, a workflow platform, a Kubernetes agent platform and two coding agents. They are collected here rather than folded into the fractions, because mixing them would make those fractions mean less, not more.

SUBJECT	CATEGORY	SPANS /RUN	GEN_AI ATTRS	SPAN NAME	KIND	STATUS	TOOL NAME	ARGS	RESULT	METRICS	ONE TRACE
Mastra	**TS framework**	**17**	**23**	✓	✓	✗	✓	✓	✓	—	✓
Goose 1.48.0	coding agent	—	17	✗	—	—	✓	✓	✓	—	✓
kagent	agent platform	67	13	✗	✓	✓	✓	—	—	—	✗
Claude Code	coding agent	—	6	✗	—	—	—	—	—	✓	✓
HolmesGPT 0.40.0	SRE agent	11	5	✗	✗	—	priv	—	—	5	✓
Langfuse 4.15.1	vendor SDK	4	0	✗	✗	—	priv	priv	priv	—	✓
n8n	workflow platform	8	0	✗	✗	✗	priv	—	—	—	✓
k8sgpt	k8s diagnostics	0	0	—	—	—	—	—	—	—	—

✓ present and specification-named · priv = present under a private name · ✗ present and wrong · — absent. Goose and Claude Code counts are attributes on their richest measured condition, not spans per run; their workload is not the harness agent. "One trace" means the whole run shares a trace id.

What the second tier says that the first does not

- **Conformance is not a language problem.** Mastra, in TypeScript, produces better GenAI telemetry than five of the seven Python setups, and is one of only two subjects anywhere in this study to emit the tool result at its defaults. The gap is a choice each project makes, not a limitation of an ecosystem.
- **The products you install are the weakest, and it is not close.** HolmesGPT, n8n and k8sgpt score 5, 0 and 0 convention attributes. You cannot fix any of them with a flag, which makes the collector the only lever you have — and only for the ones that emit something to translate.
- **Structure and vocabulary come apart.** n8n produces a perfectly shaped trace of an agent run and attaches no AI meaning to it. HolmesGPT does the reverse in its metrics, emitting two specification-named instruments while its spans use a removed attribute. Neither half implies the other.
- **An absence is a result.** k8sgpt was measured by reading its source rather than by running it: zero OpenTelemetry imports across 199 files, in the CLI and the operator alike. No amount of collector configuration reaches a process that emits nothing.

Why these are not in the fractions

Every “x of seven” in this report refers to the controlled Python cohort, where the only variable is the instrumentation. These eight vary in language, runtime, workload and category, so a ratio across all fifteen would describe the sample rather than the ecosystem. They are reported individually and cited by name.

PART 6 · IN CONTEXT

Where this work fits at Dash0

This study is a measurement of the ecosystem, not a product pitch. But the findings land directly on a set of product decisions Dash0 has already made, and it is worth being explicit about which.

The positioning, and why it matters here

In June 2026 Dash0 moved from "the OpenTelemetry-native observability platform" to **Observability for the AI Era**: a platform built so humans and agents operate software together, observing the software development harness, the agent runtime, and the full system stack. Three stages: **control what gets in**, **standardise on OpenTelemetry including AI signals**, and **reason and act**.

The middle stage is what this report tests. The claim that AI telemetry should flow through the same pipeline as everything else, under the GenAI conventions, is only as strong as the conventions' actual adoption — which is exactly what was measured.

Darkplane — the left side

Governs what AI *builds*. Observes the coding agents themselves: sessions, token usage, cost and tool calls, tying every session to the repo and branch it worked in, so a pull request arrives with its spend attached. Sessions that never produce a PR count too — that exploration is part of the bill.

Agent0 — the right side

Governs what AI *runs*. Takes a question, pulls the relevant telemetry, correlates traces, logs and metrics, follows the problem into the repository and drafts the fix as a pull request. Every tool call stays inspectable rather than producing an opaque conclusion.

Both read and write the same OpenTelemetry data layer. That is the design decision this report either supports or undermines, so it is worth stating plainly which.

What this study supports

- **Standardising on OpenTelemetry rather than a proprietary AI schema.** The alternative is correlating two systems by hand every time an agent run touches a database, which is most runs. The measurements show why: a normalized OpenInference span sits in the same trace as the HTTP request and the SQL query around it, and a proprietary AI tool's span does not.
- **Treating coding agents as ordinary instrumented services.** Goose and Claude Code both emit OTLP with no product-specific protocol. One environment variable each puts them in the same pipeline as production.
- **Reading the agent run as the unit of analysis.** This is the definition of agent observability, and the trace-shattering finding shows how many setups fail to produce one.

And what it complicates

"Every LLM call, prompt trace and agent session flows through the same pipeline" is true of the transport and not yet of the vocabulary. Five of seven libraries get span status wrong, three spellings of one finish reason coexist, only five attributes are universal, and the field that says what a tool returned is missing from six of seven. A platform reading this data has to be tolerant of all of it. That is an argument for normalising at the edge and for reading private names alongside spec names — not an argument against the strategy.

PART 6 · IN CONTEXT

The coding agent is the nearest case

The cheapest place to apply everything in this report is not production. It is the agent already running on your laptop, editing your repository.

Why this category arrived first

The industry numbers are blunt. Across 22,000 developers, incidents per pull request are up **243%** as teams lean on AI. Sixty percent say the biggest thing holding them back is a lack of clear metrics, and only **18%** measure AI's impact at all. Meanwhile a provider's billing tab gives you one number: total spend. No per-session view, no per-developer view, and no way to tell a session that shipped something from one that ran in circles for four hours.

Dash0's answer is **AI Coding Insights**, the first Darkplane capability, covering Claude Code and Cursor. It rests on four pillars:

PILLAR	THE QUESTION IT ANSWERS	WHAT YOU SEE
Cost	Where are the tokens actually going?	Spend per session, developer, repo and model, with cache savings broken out
Adoption	Is this taking hold, or stalling?	Active developers, session depth, model usage by team, WAU/MAU stickiness
Productivity	Are tokens spent efficiently?	Session turns, tool-call breakdown, cycle time from branch to merge by model
Quality	How is AI-written code holding up?	Attempts to a useful result, review comments per PR, changes requested, merge rate

The measurement problem this study found, solved in product

Claude Code emits token usage and model calls over OpenTelemetry on its own — enough for cost and adoption. This report measured exactly that base stream and found it thin: six convention attributes, no tool arguments, no tool result.

The open-source Dash0 agent plugin adds what the base stream leaves out, capturing **26 event types across 11 categories** including subagents, MCP tool calls and permission requests, as standard `gen_ai.*` and `vcs.*` traces over OTLP. That is the same gap this report documents at library level, closed by instrumenting the harness rather than waiting for the vendor.

Two details worth knowing

Cost is computed from token usage, not from the bill. Many developers are on subscriptions where the provider's dashboard shows usage as zero or heavily subsidised. Dash0 applies API-equivalent rates to the reported token counts, so you see the real cost of AI assistance even while a subscription absorbs it — which is what will actually be paid once providers move to usage-based billing.

Cost alone is a trap. A model that takes five times as many turns but costs ten times less per token looks cheap on the invoice. If those turns keep branches open twice as long, it is quietly the most expensive option available. Cost is only meaningful beside what it produced and how long it took, which is why cycle time sits next to spend rather than on its own page.

Where it sits on the ladder

LEVEL	SEEN IN	WHAT THE AI DOES
L1 · AI-assisted	GitHub Copilot	Suggests the next line
L2 · AI-generated	Claude Code, Codex, Cursor	Writes whole pull requests
L3 · AI-reviewed	CodeRabbit, Cursor Bugbot	Checks the code that AI wrote
L3.5 · AutoMerge	**Darkplane**	**Clears the safe changes on its own**
L4 · Mostly autonomous	The horizon	Runs the loop and notifies you
L5 · Dark factory	The horizon	Spec to deploy, lights-out

Most teams sit at L2, approving every pull request by hand. The work does not disappear at higher levels; it moves up a rung, from writing the change to **deciding what AI is allowed to ship unattended**. That decision needs a record of what each change cost, whether it held up, and what the agent actually did — which is the same forensic question this entire report has been asking, one layer earlier in the lifecycle.

PART 6 · COMPARISON

When to use what

The honest summary: no single choice is best, and the right one depends entirely on the question you expect to ask later.

01 **You need to know what the agent *did* — choose Pydantic AI**

It is the only setup where the tool name, its arguments and its result all arrive under specification names, on a correctly named span, inside a single trace. If your agent has write access to anything that matters, this is the only option tested that lets you reconstruct its actions afterwards.

02 **You need cost, latency and output quality — choose OpenLIT**

The richest telemetry of any non-native option: correct span naming, 2,131 characters of model text for an evaluator to read, five metrics including a dollar cost, and the specification's own event as a log record. One line to set up. Accept the missing tool result and the incorrect span status.

03 **You already run LangChain — expect to query private names**

The pairing works and does produce tool spans, but the arguments and result live in `input.value` and `output.value`, and the normalized content field contains no model text at all. Budget for writing queries against OpenInference's vocabulary rather than the specification's, and put the normalizer in your pipeline regardless.

04 **You want the specification and nothing else — choose OpenTelemetry GenAI, knowingly**

It is the most conformant implementation tested and it will tell you almost nothing about your agent at its defaults. That is not a defect: it follows the opt-in content rule the specification sets. Choose it when portability matters more than depth, and turn content capture on deliberately rather than discovering the gap during an incident.

Three things to take away

01 **The model call is solved; the agent loop is not.** Every library describes a model call competently. They diverge almost completely on describing what the agent did with the answer.

02 **Who owns the loop decides what is knowable.** A library that patches the provider SDK cannot see your tool run, cannot join your run into one trace, and no configuration changes that. It is a structural limit, not a missing feature.

03 **Conformance is not coverage.** The most specification-conformant implementation in this test records the least. A library can pass every written rule and answer none of your questions.

What we could not test

Mastra and n8n, which carry their own toolchains; Langfuse, whose SDK has no documented route to a third-party collector; convention-version sweeps, since Pydantic AI can emit under six generations and every measurement here is at defaults; and OpenTelemetry's Weaver conformance tool, which would replace this report's hand-read conformance with the project's own verdict.

Method: seven self-contained examples, three signals, real Anthropic API calls, collector-contrib 0.159.0 with the `gen_ai_normalizer` processor. Every span verified to originate from a real instrumentation library by an automated provenance gate before analysis. Versions: pydantic-ai 2.36.0, langchain 1.3.0, crewai 1.15.18, openinference-instrumentation-anthropic 2.1.1, openinference-instrumentation-langchain 0.1.73, traceloop-sdk 0.62.3, openlit 1.45.0, opentelemetry-instrumentation-genai-anthropic 1.1b1, anthropic 1.2.0 (0.125.0 for OpenLIT). Model: claude-opus-5. Measured 29–30 August 2026.

APPENDIX A

Every GenAI attribute observed, part 1

All 49 distinct `gen_ai.*` attributes captured across the seven Python setups, grouped by what they describe. A highlighted row is emitted by exactly one library. Mastra, measured separately in Part 3, adds two names that none of the seven emit: `gen_ai.tool.type` and `gen_ai.usage.reasoning_tokens`.

ATTRIBUTE (GEN_AI. PREFIX OMITTED)	PYD AI	OPENLIT	CREWAI	OPENLLM	LANGCH	OPENINF	OTEL	N
Core — emitted by every library								
`operation.name`	✓	✓	✓	✓	✓	✓	✓	7
`provider.name`	✓	✓	✓	✓	✓	✓	✓	7
`request.model`	✓	✓	✓	✓	✓	✓	✓	7
`usage.input_tokens`	✓	✓	✓	✓	✓	✓	✓	7
`usage.output_tokens`	✓	✓	✓	✓	✓	✓	✓	7
Content — the conversation								
`input.messages`	✓	✓	✓	✓	✓	✓	·	6
`output.messages`	✓	✓	✓	✓	✓	✓	·	6
`output.type`	·	✓	✓	·	·	·	·	2
`system_instructions`	·	·	✓	·	·	·	·	1
Tools — what the agent did								
`tool.args`	·	✓	✓	·	·	·	·	2
`tool.call.arguments`	✓	✓	✓	·	·	·	·	3
`tool.call.id`	✓	✓	✓	·	·	·	·	3
`tool.call.result`	✓	·	·	·	·	·	·	1
`tool.definitions`	✓	·	✓	✓	·	·	·	3
`tool.description`	·	·	·	·	✓	·	·	1
`tool.name`	✓	✓	✓	·	✓	·	·	4
Response — how it ended								
`response.finish_reasons`	✓	✓	✓	✓	·	·	✓	5
`response.id`	✓	✓	✓	✓	·	·	✓	5
`response.model`	✓	✓	✓	✓	·	·	✓	5
Request parameters								
`request.max_tokens`	·	✓	✓	✓	·	·	✓	4
`request.stop_sequences`	·	✓	✓	·	·	·	·	2
`request.stream`	·	✓	✓	·	·	·	·	2
`request.temperature`	·	✓	✓	·	·	·	·	2
`request.top_k`	·	✓	✓	·	·	·	·	2
`request.top_p`	·	✓	✓	·	·	·	·	2

Only five attributes are universal

`operation.name`, `provider.name`, `request.model`, `usage.input_tokens` and `usage.output_tokens` are the entire common ground across seven libraries. Everything else — every tool field, every content field, every cost figure — depends on which library you installed. A dashboard built only on those five works everywhere and tells you almost nothing.

APPENDIX A

Every GenAI attribute observed, part 2

ATTRIBUTE (GEN_AI. PREFIX OMITTED)	PYD AI	OPENLIT	CREWAI	OPENLLM	LANGCH	OPENINF	OTEL	N
Usage, cost and timing								
`aggregated_usage.details.cache_creation_input_tokens`	✓							1
`aggregated_usage.details.cache_read_input_tokens`	✓							1
`aggregated_usage.details.thinking_tokens`	✓							1
`aggregated_usage.input_tokens`	✓							1
`aggregated_usage.output_tokens`	✓							1
`client.operation.duration`			✓					1
`client.token.usage`		✓	✓					2
`server.time_per_output_token`		✓	✓					2
`server.time_to_first_token`		✓	✓					2
`usage.cache_creation.input_tokens`		✓	✓	✓			✓	4
`usage.cache_read.input_tokens`		✓	✓	✓			✓	4
`usage.cost`		✓	✓					2
`usage.details.cache_creation_input_tokens`	✓							1
`usage.details.cache_read_input_tokens`	✓							1
`usage.details.thinking_tokens`	✓							1
`usage.total_tokens`				✓				1
Agent and workflow identity								
`agent.call.id`	✓							1
`agent.description`			✓					1
`agent.id`			✓					1
`agent.name`	✓		✓					2
`conversation.id`	✓							1
`workflow.name`			✓					1
Other								
`sdk.version`		✓	✓					2
`system`	✓							1

What the shape of this table says

- **Pydantic AI keeps its own usage namespace.** It reports `usage.details.thinking_tokens` and a parallel `aggregated_usage.*` family that no other library emits. Richer, and not portable.
- **OpenLIT and CrewAI dual-emit tool arguments.** Both `tool.args` and `tool.call.arguments` appear on the same span — a deliberate migration overlap, documented in OpenLIT's source.
- **OpenLLMetry emits `usage.total_tokens`**, which the conventions do not define. The spec has input and output only.
- **CrewAI contributes the richest agent identity:** `agent.id`, `agent.description`, `workflow.name` and `system_instructions`, none of which appear anywhere else.
- **OpenTelemetry's own instrumentation emits the fewest,** and every one it emits is emitted by at least four others. It is the intersection, not the union.

APPENDIX B

Where the reasoning goes

Modern models think before answering. Tracking where that reasoning ends up produced the most counter-intuitive result in this study, and it is not the libraries' fault.

Three libraries record that reasoning happened

LIBRARY	WHERE IT APPEARS	WHAT IS ACTUALLY IN IT
Pydantic AI	`gen_ai.output.messages` → `parts[].type = "thinking"`	The typed part is present. **Content is empty.**
OpenLLMetry	`gen_ai.output.messages` → `parts[].type = "reasoning"`	The typed part is present. **Content is empty.**
OpenInference	`llm.output_messages.N.message.contents.M.message_content.type = "reasoning"`	A 520-character `signature`. The sibling `.text` is **empty**.
OpenLIT, CrewAI, LangChain, OTel GenAI	—	Nothing at all.

The reasoning is empty at the source, not lost in the telemetry

Claude Opus 5 defaults its thinking `display` to `omitted`, which returns thinking blocks with empty text. Every library above is faithfully recording an empty block. **No instrumentation setting would fix this** — the content never left the provider.

Recovering it requires changing the *request*, not the instrumentation: asking for `thinking: {type: "adaptive", display: "summarized"}`. None of the seven libraries surfaces that choice, warns that the block it recorded is empty, or distinguishes "the model did not think" from "the model thought and did not tell you".

Only one library tells you reasoning happened at all

Pydantic AI reports `gen_ai.usage.details.thinking_tokens` — measured between 61 and 83 tokens per run. That is the only quantitative signal in the entire study that reasoning occurred and how much of it. Every other setup gives you no way to distinguish a run where the model reasoned extensively from one where it answered immediately, even though the two have different costs and different reliability characteristics.

Why this matters more than it looks

Reasoning tokens are billed. They also correlate with the difficulty the model encountered, which is exactly the signal you would want when investigating why one run behaved differently from another. In this study, the agent's decision to decline a destructive action was a reasoning outcome — and in six of seven setups the telemetry gives no indication that any reasoning took place.

What would close this

Two things, neither of them a new attribute. First, instrumentation should record the thinking-token count wherever the provider returns one — Pydantic AI shows it is available. Second, a captured thinking block with empty content should be distinguishable from an absent one, so an operator can tell that reasoning happened and was withheld rather than assuming the model did not reason.